Laboratory Report

I

LECTURER	John ORaw
STUDENT NAME	Edmund Connolly
STUDENT NUMBER	L00194727
PROGRAMME	MSc Cloud
MODULE CODE	IaC
ASSIGNMENT TITLE	Week 7 Powershell
SUBMISSION DATE	16 Nov 2025

Student Declaration

1. I have accurately identified and included the sources of all facts, ideas, opinions, and viewpoints from others in the assignment references. All direct quotations, paraphrasing, and discussions of ideas from books, journal articles, internet sources, course materials, or any other sources used are properly acknowledged and cited in the assignment references.
2. I have not used unauthorised artificial intelligence tools or aids.
3. I understand and am compliant with ATU's policy and procedures regarding Academic Integrity and I am aware of the consequences of any violations.
4. I have followed the referencing guidelines recommended in the assignment instructions and / or programme documentation.
5. By signing this form or by submitting material online I confirm that this assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other programme of study.
6. By signing this form or by submitting material for assessment online I confirm that I have read and understood AQAE022 Academic Integrity Policy

Student Signature	**Date**
Edmund Connolly	16 Nov 2025

Contents

Table of Figures

Description

Windows Operating Systems is the predominant operating systems for personal desktop users, and is also commonly used for servers. Windows PowerShell is the new preferred method for scripting on windows machines, one major advantage of PowerShell is it provides cross-platform support across Windows, Linux and macOS machines. This makes it essential for system administrators to learn and become proficient with.

Aims

PowerShell scripts can be used across Windows, Linux and macOS machines by system administrators to automate repetitive time-consuming tasks and these tasks can be scheduled to run at certain times and a regular basis.

1. Become familiar with writing, testing and executing PowerShell scripts.
2. To install latest version of PowerShell Version 7.
3. To install Visual Studio Code PowerShell extension.
4. To become familiar with using Visual Studio Code editor and PowerShell.
5. Observe the output from executing the PowerShell script files.

Method

For writing, testing and executing our PowerShell scripts I will use a Microsoft Windows 10 Virtual Machine. Initially I will use The PowerShell ISE but later will download the newer PowerShell version 7.x and use with Visual Studio Code. Visual Studio Code has a PowerShell extension to help write, test and debug PowerShell.

1.Setup.ps1

The code for 1. Setup.ps1 is below.

```
# Check the existing version
$PSVersiontable
# Set an execution policy
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Force
# Install Nuget as a package provider
Install-PackageProvider Nuget -MinimumVersion 2.8.5.201 -Force | Out-Null
# Install the module
Install-Module -Name PowerShellGet -Force -AllowClobber
# Create a script directory
mkdir c:\PowerShell
```

The code first checks the existing version of PowerShell I am using, it then will try set the execution policy to forced remote signed, this would allow me to run scripts I write and also ones I download but they must be digitally signed by a trusted publisher. It installs a package manager Nuget which must not have a version number less than 2.8.5.201. Then it installs the PowerShellGet module. Lastly it creates a directory PowerShell on the c drive.

2. Download PowerShell7.ps1

The code for 2. Download PowerShell7.ps1 is below.

```
# Download PowerShell 7 installation script
Set-Location C:\PowerShell
$URI = "https://aka.ms/install-powershell.ps1"
Invoke-RestMethod -Uri $URI |
Out-File -FilePath C:\PowerShell\Install-PowerShell.ps1
```

The code firsts sets the location to the PowerShell folder in the root C: drive.
In the 2nd line it sets a PowerShell variable named URI to the Location specified by Uniform Resource Identifier link where the latest version of PowerShell is located.
In the 3rd line I then download the latest version using Invoke-RestMethod command and specifying the Uri is held in the $URI variable, the output is piped using | into a file called Install-Powershell, this creates a file that is a local installer of PowerShell.

3. Install-PowerShell.ps1

The code for 3. Install-PowerShell.ps1 is below.

```
$MYPARMS = @{
    UseMSI = $true
    Quiet = $true
    AddExplorerContextMenu = $true
    EnabePSRemoting = $true
}
C:\PowerShell\Install-PowerShell.ps1 @MYPARMS
```

The code creates a variable called MYPARMS, which contains settings for installing the new version of PowerShell.
UseMSI = $true instructs PowerShell to use the Microsoft Installer.
Quiet = $true instructs Powershell to not prompt and to not need manual confirmation from users.
AddExplorerContextMenu = $true, adds a Open PowerShell here to the menu that appears when you right-click in File Explorer
EnablePSRemoting = $true allows PoweShell Remoting.

The final command tells PowerShell to run the script file Install-PowerShell.ps1 located in the PowerShell Folder with the settings in MYPARMS.

4. Verify PowerShell.ps1

I will now use Visual Studio code as this is the default development tool for PowerShell 7. Open Visual Studio code and install the PowerShell extension, by going to Extensions and from the list install PowerShell extension.

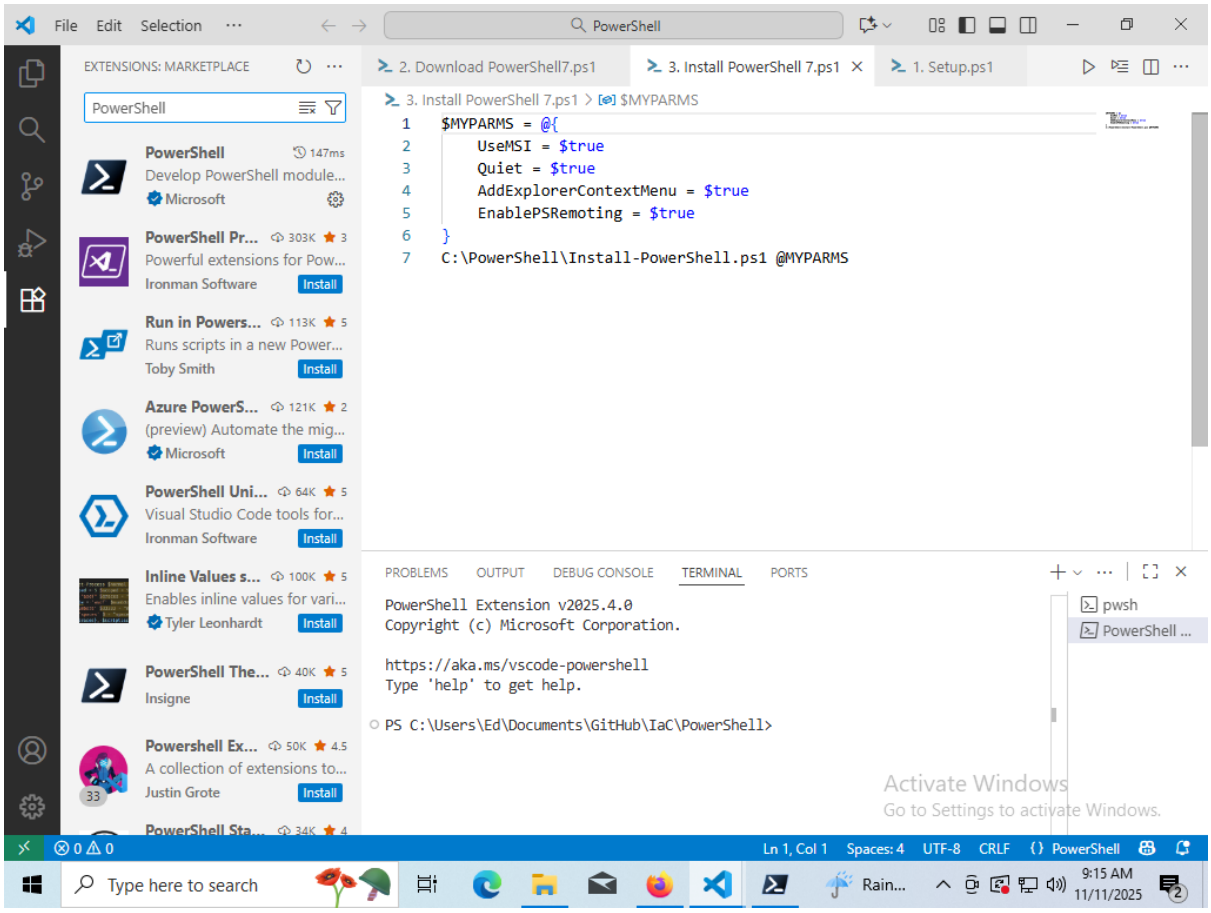

Figure 1 - Installing PowerShell Visual Studio Code Extentsion

The code for 4. Verify PowerShell.ps1 is shown below

```
$I = 0
$env:PSModulePath -split ';' |
Foreach-Object {"[{0:N0}] {1}" -f $I++, $_}
```

The code sets a variable $I = 0 initally. Then it uses a function to split and breakup the pipes specified by the pipe symbol |. The final result is the locations of the modules PowerShell uses.

5. FindModules.ps1

The code for 5. FindModules.ps1 is shown below:

```
$PGSM = Find-Module -Name *
"There are {0:N0} Modules in the PS Gallery" -f $PGSM.count
```

The variable $PGSM is assigned all modules which match the specified pattern, The pattern is -Name * which is find all modules.

The 2nd line prints out the total number of modules, it uses format operator -f, and {0:N0}, 0 is the count of modules and N0 is a numeric formatter for a thousand separator and zero decimal places.

6. FindModulesPSCore.ps1

The code for 6. FindModulesPSCore.ps1 is below:

```
$PGSMC = Find-Module -Name * -Tag 'PSEdition_Core'
"There are {0:N0} Modules that Support PowerShell Core" -f $PGSMC.Count
```

The code finds all modules that have a “PSEdition_Core” tag and assign the result to the variable $PGSMC. On the 2nd line I print out the total number of modules which have the tag “PSEdition_Core”, it uses format operator -f, and {0:N0}, 0 is the count of modules and N0 is a numeric formatter for a thousand separator and zero decimal places.

At this stage I have PowerShell7.x installed and the Visual Studio development environment also works.

Coding in PowerShell

Creating a module

I now proceed to create a PowerShell module. The following code was entered into the VS Code terminal

```
$MyModulePath =
"C:\Users\$env:USERNAME\Documents\PowerShell\Modules\HelloWorld"

$MyModule = @"
# HelloWorld.PSM1
Function Get-HelloWorld {
 "Hello World from JOR"
}
"@

New-Item -Path $MyModulePath -ItemType Directory -Force | Out-Null
$MyModule | Out-File -FilePath $MyModulePath\HelloWorld.PSM1
Get-Module -Name HelloWorld -ListAvailable
```

The code stores the path where the new module will be stored in a variable called MyModulePath, it uses the environment username variable ($env:USERNAME). It then creates a variable called MyModule which will contain my new module called HelloWorld.PSM1, this module has a comment # HelloWorld.PSM1 and also defines a Function that returns the string “Hello World from JOR”

```
New-Item -Path $MyModulePath -ItemType Directory -Force | Out-Null
```

This creates the directory specified in the MyModulePath variable if it doesn’t exist.
The – Force switch makes PowerShell create any missing parent directories if they don’t exist
The output is pipped with the | pipe operator to Out-Null, to not display the output to the screen.

```
$MyModule | Out-File -FilePath $MyModulePath\HelloWorld.PSM1
```

This saves the module to the proper location and names it HelloWorld.PSM1

```
Get-Module -Name HelloWorld -ListAvailable
```

This checks is there a module named HelloWorld available in PowerShell after creating it.

Variables

The command Get-Variable is entered into the PowerShell terminal, it displays all the currently defined variables and their value in the current session.

Then I enter the following code into the terminal:

```
$Rubbish = 1, 2, "a", "ff"
$Rubbish
clear-variable -Name Rubbish
$Rubbish
Remove-Variable -Name Rubbish
```

This code sets the variable $Rubbish equal = 1, 2, “a”, “ff”, it then prints it out to the console, followed by clearing the value, I then print its current value out and finally remove the variable Rubbish.

Next, I look at a variables type, I can store any type of object in a variable, such as arrays, integers and strings. I use the following code to investigate types:

```
$Rubbish = 1, 2, "a", "ff"
$Rubbish.GetType()
```

The code sets the rubbish value, and then I look at the type of variable that Rubbish has been assigned based on the variable type it got.

Next, I examine casting, when I cast a value I am basically specifying what type it is set to. I used the following code to set a variable called Rubbish to an int type, then I check what type the Rubbish value is by using GetType function.

```
[int]$Rubbish = 1
$Rubbish.GetType()
```

Automatic conversion of datatypes is performed by PowerShell, I set the variable rubbish = 1 and cast it as an int, then I set it to the string “123456789”.

```
[int]$Rubbish = 1
$Rubbish = "123456789"
$Rubbish
```

Automatic conversion failed for the following code due to the string containing nonnumeric values

```
[int]$Rubbish = 1
$Rubbish = "This will give you an error!"
$Rubbish
```

The next code snippet creates a variable called OGGI which contains the string variable 10/13/2025 and is then cast to a type of datetime, I then print out the resultant variable.

```
[datetime]$OGGI = "10/13/2025"
$OGGI
```

The code for getting PowerShell to accept dates in DD/MM/YY format is shown next.

```
$DDMMYY = "13/10/2025"
$OGGI = [datetime]::ParseExact($DDMMYY, "dd/MM/yyyy",
[System.Globalization.CultureInfo]::GetCultureInfo("en-IE"))
```

It sets a variable called DDMMYY, which is a string in day month year format, I then use the ParseExact function in the datetime class, ParseExact is passed the date string, then the format I expect i.e DD/MM/YY , the last parameter is the culture information which is set to English-Ireland.

Next I look at storing the result of a command in a variable, the code below stores the result of the Get-ChildItem c:\ this is the equivalent of the directories in the root c drive. It then displays the list stored in the variable to the screen.

```
$dir_listing = Get-ChildItem c:\
$dir_listing
```

To check if a variable called EdsVariable exists I can use the following code. First I create the variable EdsVariable with a value of 3.142 and a description, I set it to read only to make it constant. Then I check it

```
New-Variable EdsVariable -value 3.142 -description "PI with write-protection"
-option ReadOnly
Get-Variable EdsVariable
```

Finally to preform a simple tax calculation I implement the code below, I set the net and vat variables, then when calculate the vatamount by mulitpling the amount by the vat rate, then I add the vatamount to the net and print the result.

```
$NET = 111
$VAT = 0.23
$VATAMOUNT = $amount * $VAT
$GROSS = $NET + $VATAMOUNT
$text = "The total €$GROSS is the sum of the net value €$amount with the VAT
amount €$VATAMOUNT at $VAT% VAT rate"
$text
```

Types

The Types code exercises are shown below, the first snippet below stores the string Yoo hoo! In the variable called StringValue, then it calls the ToUpper() function on StringValue, this results in all the letters being converted to upper case i.e. capitalised. The ToLower() function converts all the letters to lower case i.e. not capitalised.

```
$StringValue = "Yoo hoo!"
$StringValue.ToUpper()
$StringValue.ToLower()
```

The code below defines an array called MyArray and populates it with the values 1,2,3,4,5. When I access $MyArray[1] I am accessing the 2nd element of the array because the indexing of the array begins at 0

```
$MyArray = 1,2,3,4,5
$MyArray[1]
```

The code below declares a variable called LittleNumber with value 12345, it then prints its type. It also declares a Bignumber with value 123456789123456789, and prints its type. I can see LittleNumber is type Int32 whereas Bignumber is int64

```
$LittleNumber = 12345
$LittleNumber.GetType()
$BigNumber = 123456789123456789
$BigNumber.GetType()
```

The code below declares a variable called Floaty32 with value 12.12, it then prints its type. It also declares a Float64 with value 12345.1234, and prints its type. I can see Floaty32 is type Single whereas Float64 is double

```
[float]$Floaty32 = 12.12
$Floaty32.GetType()
[double]$Floaty64 = 12345.1234
$Floaty64.GetType()
```

Conditional Branching

The code for conditional branching uses if statements. The snippet below first sets 2 variables, variable1 = 32 and variable2 = 33, it then prints out the “The condition was true “ if variable 1 is not equal to variable 2

```
$Variable1 = 32
$Variable2 = 33
if ( $Variable1 -ne $Variable2  )
{
    Write-Output "The condition was true"
}
```

The code for conditional branching below uses if and elseif statements to convert a numeric value to the equivalent day of the week. The snippet below first sets a variable day = 3, it then goes into the if else block and finds where day variable equals 3 and sets the $result to the corresponding day, it then prints out the day.

```
$day = 3

if ( $day -eq 0 ) { $result = 'Sunday'        }
elseif ( $day -eq 1 ) { $result = 'Monday'    }
elseif ( $day -eq 2 ) { $result = 'Tuesday'   }
elseif ( $day -eq 3 ) { $result = 'Wednesday' }
elseif ( $day -eq 4 ) { $result = 'Thursday'  }
elseif ( $day -eq 5 ) { $result = 'Friday'    }
elseif ( $day -eq 6 ) { $result = 'Saturday'  }

$result
```

The code example for conditional branching below sets a variable it then uses if statement with the -like tag to check if the string begins with '$SD', it also checks if Ed occurs anywhere in the string and finally it checks does the string end with TCH.

```
$FINDVALUE = '$SDDPT,2.98,,*66, Edmund'
if ( $FINDVALUE -like '$SD*')
{
    Write-Output "Found a depth sounder"
}
if( $FINDVALUE -like '*Ed*')
{
    Write-Output "Found Ed"
}
if ( $FINDVALUE -like '*TCH' )
{
    Write-Output "No match found"
}
```

The code for conditional branching below uses a switch statement to convert a numeric value to the equivalent day of the week. The snippet below first sets a variable day = 4, it then goes into the switch statement finds where day variable equals 4 and sets the $result to the corresponding day, it then prints out the day.

```
$day = 4
```

```
switch ( $day )
{
    0 { $result = 'Sunday'    }
    1 { $result = 'Monday'    }
    2 { $result = 'Tuesday'   }
    3 { $result = 'Wednesday' }
    4 { $result = 'Thursday'  }
    5 { $result = 'Friday'    }
    6 { $result = 'Saturday'  }
}

$result
```

Loops

For Loop

The code for a for loop is shown below, it initialises the counter value to 0, and executes the statements in the body of the for loop while the counter is less than 10, each time around the for loop it increments the counter.

```
for ($counter = 0; $counter -lt 10; $counter++)
{
    $counter
}
```

ForEach

The for each loop is used to loop through an array. It loops through every element of an array until the end. The code below is an example of a for each loop.

```
$MyArray = "J", "o", "h", "n"
foreach ($Letter in $MyArray)
{
    $Letter
}
```

While

The while loop is used to loop until the condition being evaluated is not true. In the code snippet below, the code body of while loop which increments and prints out the $val variable will execute as long as $val is less than 3.

```
$val = 0
while($val -lt 3)
{
    $val++
    Write-Host $val
}
```

For the next while loop example, the code is shown below. It takes input from the user and stores it in a variable called $inp, the loop continually executes while the input is not equal to “Q”. In the Switch statement I check the value of the input if it’s:
L the following will be printed "File will be deleted"
A the following will be printed "File will be displayed"
R the following will be printed "File will be write protected"
Q the following will be printed "End"

The default message "Invalid entry" when the input is not defined i.e not L, A, R, or Q.

```
while(($inp = Read-Host -Prompt "Select a command") -ne "Q")
{
    switch($inp){
       L {"File will be deleted"}
       A {"File will be displayed"}
       R {"File will be write protected"}
       Q {"End"}
       default {"Invalid entry"}
   }
}
```

Do Until

The Do until code snippet below, is similar to the while loop except it always get executed at least 1 time, and it is condition is evaluated at the end of the loop and doesn’t exit until the condition is true. The snippet below sets a variable $a = 0, then it loops, in the loop it prints out the current value then increments it and prints out the new value, it does this until the value is greater than 5.

```
$a = 0
do
{
    “Starting Loop $a”
    $a
    $a++
    “Now `$a is $a”
} until ($a -ge 5)
```

Piping Commands

The code below performs piping of the Dir command which lists the files and directories in the current directory, it pipes this into format table which takes the output from DIR as input and formats it into a table with columns for different values it then pipes this to out-host which sends it to the console.

```
Dir | Format-Table | Out-Host
```

Results and Testing

The results of my PowerShell scripts are shown here, they were collected by running the PowerShell scripts on a Windows virtual machine.

1.Setup.ps1 Results

The result for execution of 1. Setup.ps1 Result is shown in the appendix as Figure 2.

I can see from the screenshot

```
Name                           Value
----                           -----
PSVersion                      5.1.26100.7019
PSEdition                      Desktop
PSCompatibleVersions           {1.0, 2.0, 3.0, 4.0...}
BuildVersion                   10.0.26100.7019
CLRVersion                     4.0.30319.42000
WSManStackVersion              3.0
PSRemotingProtocolVersion      2.3
SerializationVersion           1.1.0.1
```

This tells us I am using PowerShell 5.1 and that the PSEdition is the desktop edition, I can see the version number value for other things as well. The Execution Policy was updated successfully otherwise I would receive error or warning messages in red typeface in the terminal. The PowerShell ISE displayed a popup, showing the name of the module being installed and a progress bar. In figure 3, I can see that the PowerShell Folder has been created in the Windows Virtual Machine C:

2. Download PowerShell7.ps1 Results

The result of executing PowerShell7.ps1 is shown in figure 4, I can see in the Windows Virtual Machine C:\PowerShell folder there now is a local installer file that contains the data necessary to install the latest version of PowerShell.

3. Install PowerShell7.ps1 Results

The result of running 3. Install PowerShell7.ps1 can be seen in figure 5. Execution of the script showed displayed a popup which showed files downloading from GitHub. In figure 5, I can see the PowerScript ran without error.

4. Verify PowerShell.ps1 Results

The result of running 4. Verify PowerShell.ps1 is shown in figure 6 and figure 7. Figure 6 shows what modules PowerShell uses, and figure 6 shows that I am using the latest version of PowerShell that I just installed.

5. FindModules.ps1 Results

The results of running 5. FindModules.ps1 is shown in figure 8,I can see in the terminal that there are 3,160 Modules in the Power Script Gallery.

6. FindModulesPSCore.ps1 Results

The resultant number of modules is shown in figure 9, it says there are 0 modules.

Coding in PowerShell

Creating a module

In figure 10, I can see there is now a HelloWorld.PSM1 module available in PowerShell listed and in figure 11, I can see the HelloWorld PowerShell module is in the directory specified and it also contains the code I wrote for it. In figure 12 I can see it prints out the message in the terminal.

Variables

In figure 13 I can see the output of Get-Variable which lists all the variables that exist in the current session, each variable is listed with its name and its current value.

In figure 14 I can see the result of setting the value $Rubbish, printing out its current value, then I clear the variable and printing out the variable shows it has been cleared, finally I remove the variable.

In figure 15 I can see the result of using the function GetType(), it states that it is an array of objects. And that's it is public and is serial. Its an array because it is storing 4 values.

In figure 16, I note the rubbish value has a type of Int32 which is integer type with a size of 32 bits because of the casting operation.

For figure 17, I can see that PowerShell has automatically converted $Rubbish from the string "123456789" to its int value.

In figure 18 I can see that PowerShell hasn't been able to do the conversion from string to int, due to the string containing non numeric values.

In figure 19 I can see that casting the string 10/13/2025 to a datetime object gives us a resultant formatted datetime string.

In figure 20, I can see the datetime module now accepts dates in the format DD/MM/YY.

In figure 21, I can see output of the Get-ChildItem c:, the output of which is stored in the variable $dir_listing. This is the equivalent of the directoried in the root c drive.

In figure 22 I can see that it shows the EdsVariable exists after I created it and what its value is.

In figure 23 I can see that the tax calculator prints out the result of the calculation based on the variables values that were set.

Types

In figure 24 I can see the results from running the Types code exercises.

ToUpper() function on StringValue, results in all the letters being converted to upper case i.e. YOO HOO! whereas The ToLower() function converts all the letters to lower case yoo hoo.

The code for the array called MyArray and populates it with the values 1,2,3,4,5. When I access $MyArray[1] I get the value 2 because index 0 is the first element

The code which declares a variable called LittleNumber with value 12345, has type Int32 because it can fit in a integer with 32 bits. It also declares a Bignumber with value 123456789123456789, its type is int64, a larger integer size is needed to store the bignumber because a 32 bit int can store +/- 2^{31} values.

The code that declares a variable called Floaty32 with value 12.12, has type float32. It also declares a Float64 with value 12345.1234, and which has a float32 type. A `A 32 bit float is required to be able to store the smaller fraction of 12345.1234 with more precision.`

Conditional Branching

In figure 25 I see the result of comparing two variables to check if they are not equal, as variable1 = 32 and variable2 = 33 the if condition is true, they are not equal and therefore “The condition was true” was printed out.

The next bit of code converts a numeric date, i.e. the day value of the week to the day name. It sets a day variable to 3, then in the if else statement $day = 3 thus the $result variable is set to Wednesday. Then at the end it prints out the result variable which is “Wednesday”.

In the code example, it uses if statement with the -like tag to check if the string begins with ‘$SD’ which it does so "Found a depth sounder" is printed out, it also checks if Ed occurs anywhere in the string which it does so it prints out “Found Ed”, and finally it checks does the string end with TCH which it doesn’t so it doesn’t print out anything.

The final code for the conditional branching section uses a switch statement to convert a numeric value to the equivalent day of the week. The snippet below first sets a variable day = 4, it then goes into the switch statement finds where day variable equals 4 and sets the $result to the corresponding day, it then prints out the day which is Thursday.

Loops

For Loop

The result of running the for loop code is shown in figure 27, the $counter value is printed to the screen, it initially begins at 0, prints the current counter value, increments the current counter value by 1, then a check is made to is if the counter is less than 10, this for loop repeats the process until the counter is not less then 10.

ForEach Loop

The result of running the for each loop is shown in figure 28. Its starts at the beginning of the array and assigns the $letter variable with the current index element of the array which is the 0^{th} element $letter is assigned “J” then it moves to the next element and assigns the next element to the letter variable and prints it out it repeats this process until it reaches the end.

While Loop

The results for the while loop code is shown in figure 29. The first code snippet initalises $val to 0, it then enters a while loop that checks the condition which is $val is less than 3, as it is less than 3 it executes the loop body which increments the $val variable to 1 and then prints it out to the screen this process repeats until $val is not less than 3. Thus the printed out values are 1,2,3.

The 2nd code snippet continually reads input form the user, and this input is the condition to the while loop, the while loop continues to execute until the letter Q is entered, inside the while loop is a switch statement which evaluates the users input based on the input it does the following

L the following will be printed "File will be deleted".
A the following will be printed "File will be displayed".
R the following will be printed "File will be write protected".
Q the following will be printed "End".

Do Until

The result of the do until loop can be seen in figure 30, I begin by setting $a = 0, then I loop until $a is greater than 5, every time I loop I increment the value of $a and I print out its value before and after incrementing.

Piping Commands

The result of piping can be seen in figure 31, the list of directories and files is formatted by piping the dir command into the Format-Table, finally the output is presented on the screen by piping to Out-Host.

Conclusions

It should be confirmed if the aims have been met, based on the results or testing. Evidence of independent research should be provided and cited from the text. The conclusion should show an understanding of why the work was significant. Most marks go for the conclusion, this section should be substantial.

In this laboratory all the aims were met I successful, wrote, tested and executed PowerShell scripts. I updated and installed the latest version of PowerShell v7 and installed and configured the PowerShell extension for visual studio code. I verified the installation of newer version by writing and executing a PowerShell script which returned what version of PowerShell our system was using. The work is important as it sets the foundations for enabling the reader to read and write base PowerShell scripts that can automate long and tedious processes that can be of use to system administrators across Linux, MacOS and Windows platforms.

PowerShell is more like a programming language then a command-line program. It uses objects, in fact everything in PowerShell is an object. The objects have properties and methods which are used to represent attributes and instructions [1]. It has 4 different types of commands:

1. Cmdlets these are basic single-function commands of PowerShell, they are not written in PowerShell, they can be combine to carry out more significant functions and they may be piped using the | character
2. PowerShell functions are written in the PowerShell language, you form a sequence of instructions and then invoke the function to execute it, the output can be displayed on the screen or piped to another function or cmdlet input
3. PowerShell scripts are written using cmdlets, there are 3 types of command
    - Get command is used to retrieve data from file system.
    - Set command is used to edit the windows component info.
    - Remove command is used to delete operations completely.

4. Executable commands are executable files and have a .exe file type extension and they are components of Windows

One of the first experiments is with PowerShell Modules PowerShell modules are a self-contained, reusable unit that can include cmdlets, providers, functions, variables and other resources [2]. As stated the main advantage of PowerShell's Modules is they are reusable this is good as they can be reused on other machines. Some advantages are, I have less repetition as I write code only once, because the script is modularised it is easier to maintain. Modules allow the scripter to better organise their work, it keeps the related functions and variables together in the module. They are easier to share with others as modules can be packed and distributed so others can use them immediately. Lastly, they are scalable for larger projects.

I also experimented with some basic PowerShell features such as how to define, set and clear different variables, during this process I learnt that variables in PowerShell have a "type". Variables are used to store objects, but these objects can have types, such as String, Int, Int32, Float, Bool [3]. Strings are used to store characters, numbers and special characters, whereas Int is used to store whole numbers, Float can store numbers with fractions, and a Boolean can store true or false. There can be issues with doing operations with different types, such as adding a string to an int or adding an int to a string, thus it maybe necessary to cast the data types first to get the desired output. [3]

I also experimented with PowerShell's different types of loop; While, For, ForEach, Do until and also investigated conditional statements. These programming constructs allow programmers/scripters to execute code a fixed number of times with a for loop, or while a condition holds true using a while loop. Then using an if statement and switch statements we can execute code based on the current value of a variable or an inputs value.

References

Any external research referenced should be documented here, in an accepted format. The Institute standard is Harvard, I prefer IEEE referencing for short papers and reports. Either is acceptable but be consistent.

[1] C. BasuMallick, "What Is PowerShell? Meaning, Working, Uses, and Advantages |," *Spiceworks*, Aug. 26, 2022. https://www.spiceworks.com/tech/devops/articles/what-is-powershell/

[2] sdwheeler, "about_Modules - PowerShell," *Microsoft.com*, Oct. 10, 2024. https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.core/about/about_modules?view=powershell-7.5

[3] A. Cobar, "PowerShell Variable Examples for Data Types, Scope, Naming, Assignment," *MSSQLTips.com*, Feb. 10, 2022. https://www.mssqltips.com/sqlservertip/7153/powershell-variable-examples-data-types-scope-name-assignment/ (accessed Nov. 23, 2025).

Appendices

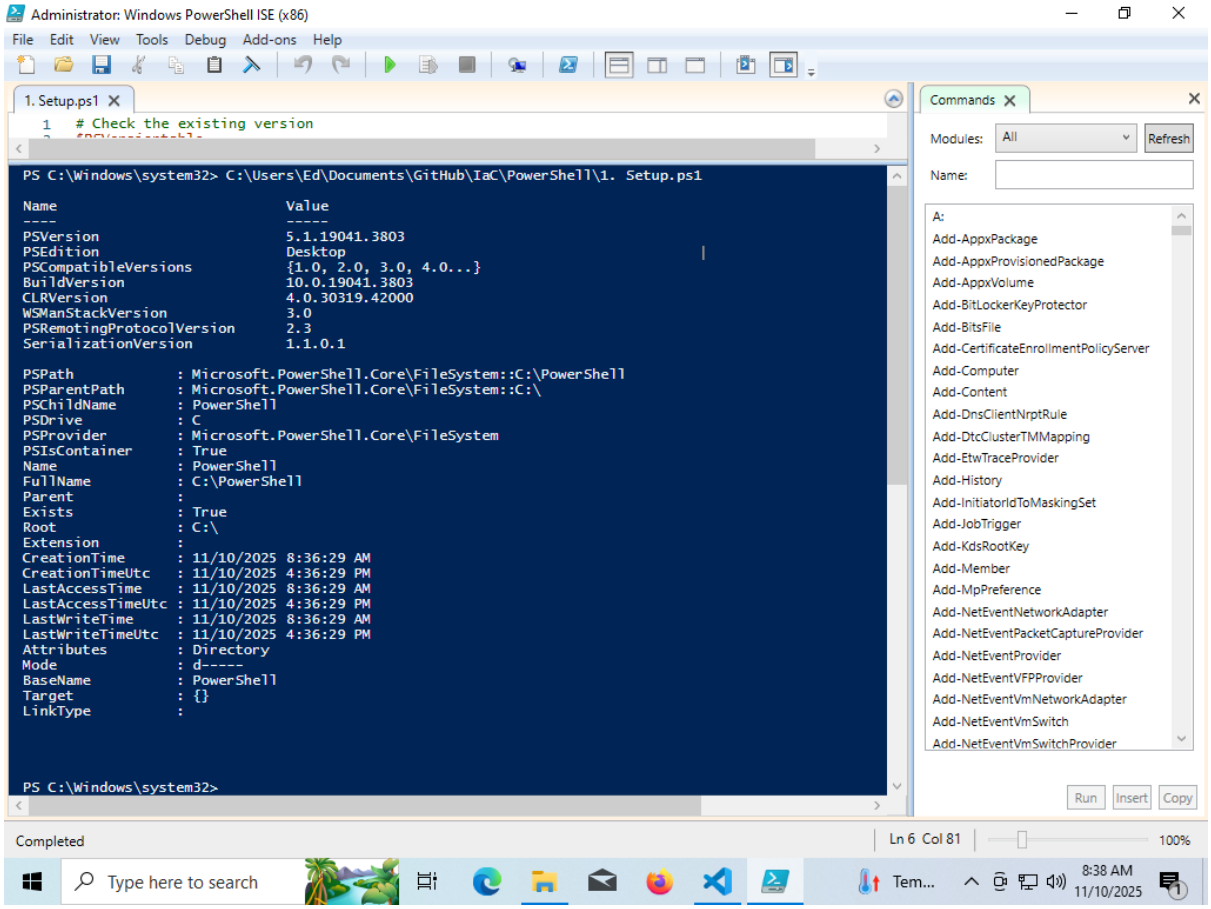

Figure 2- Result of Running Script 1. Setup.ps1

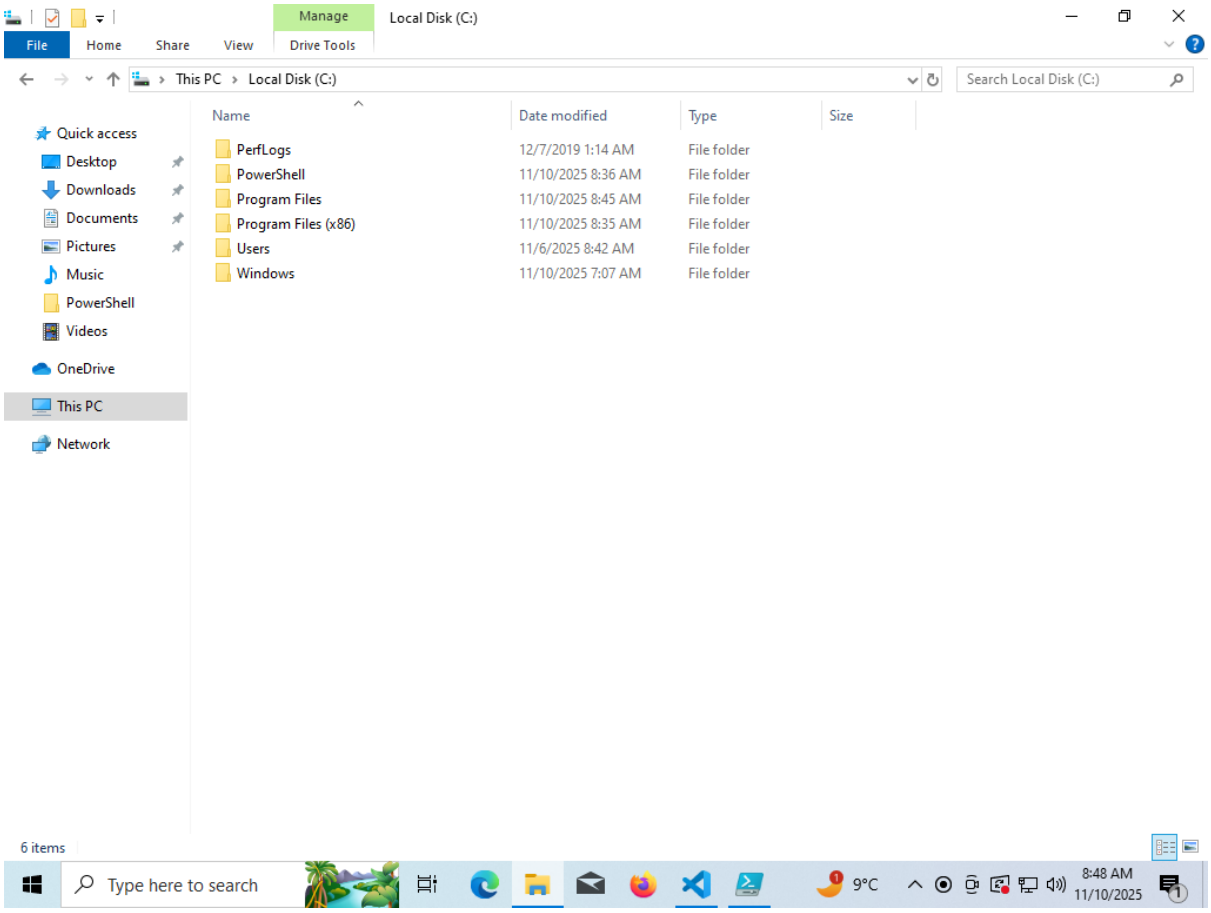

Figure 3 – Creation of Powershell Folder

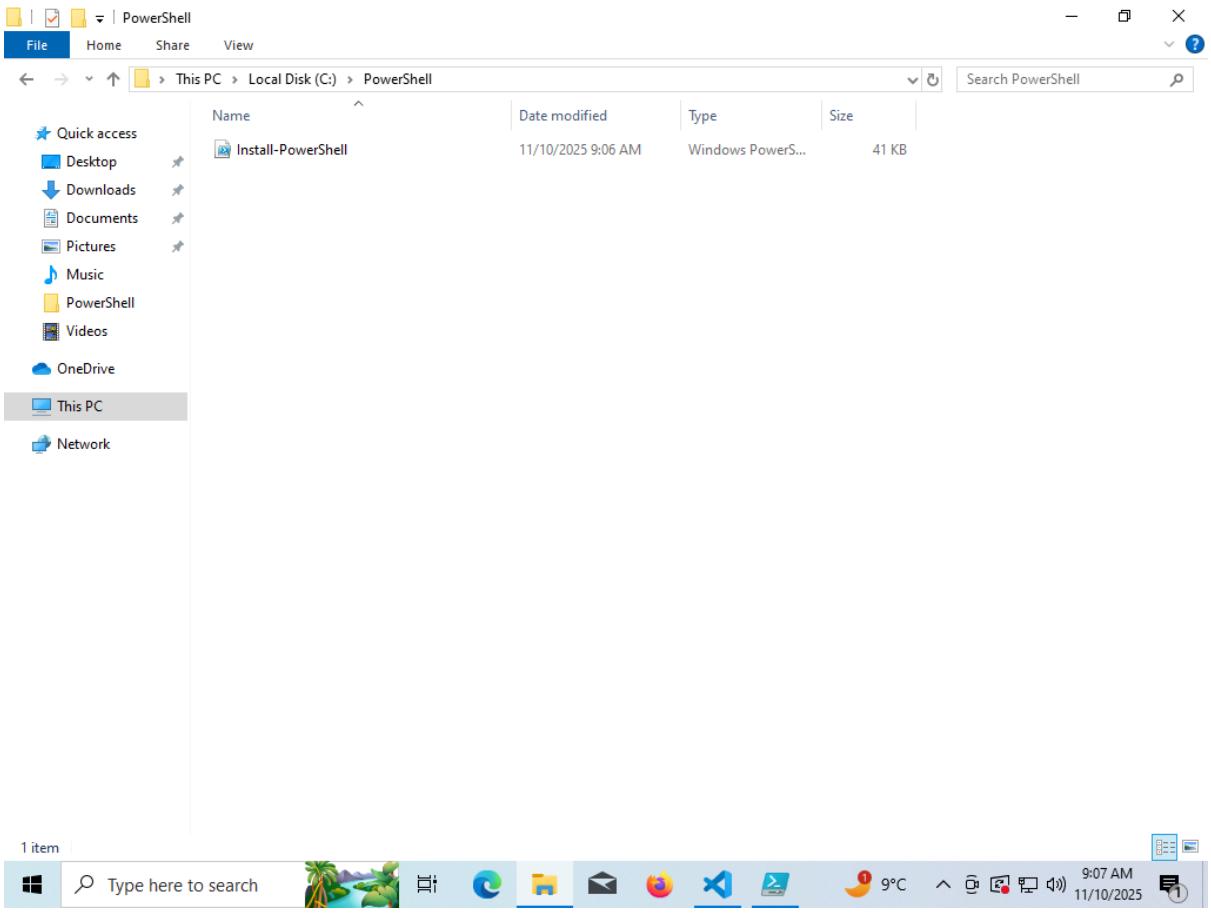

Figure 4 – Result of 2. Download PowerShell7.ps1

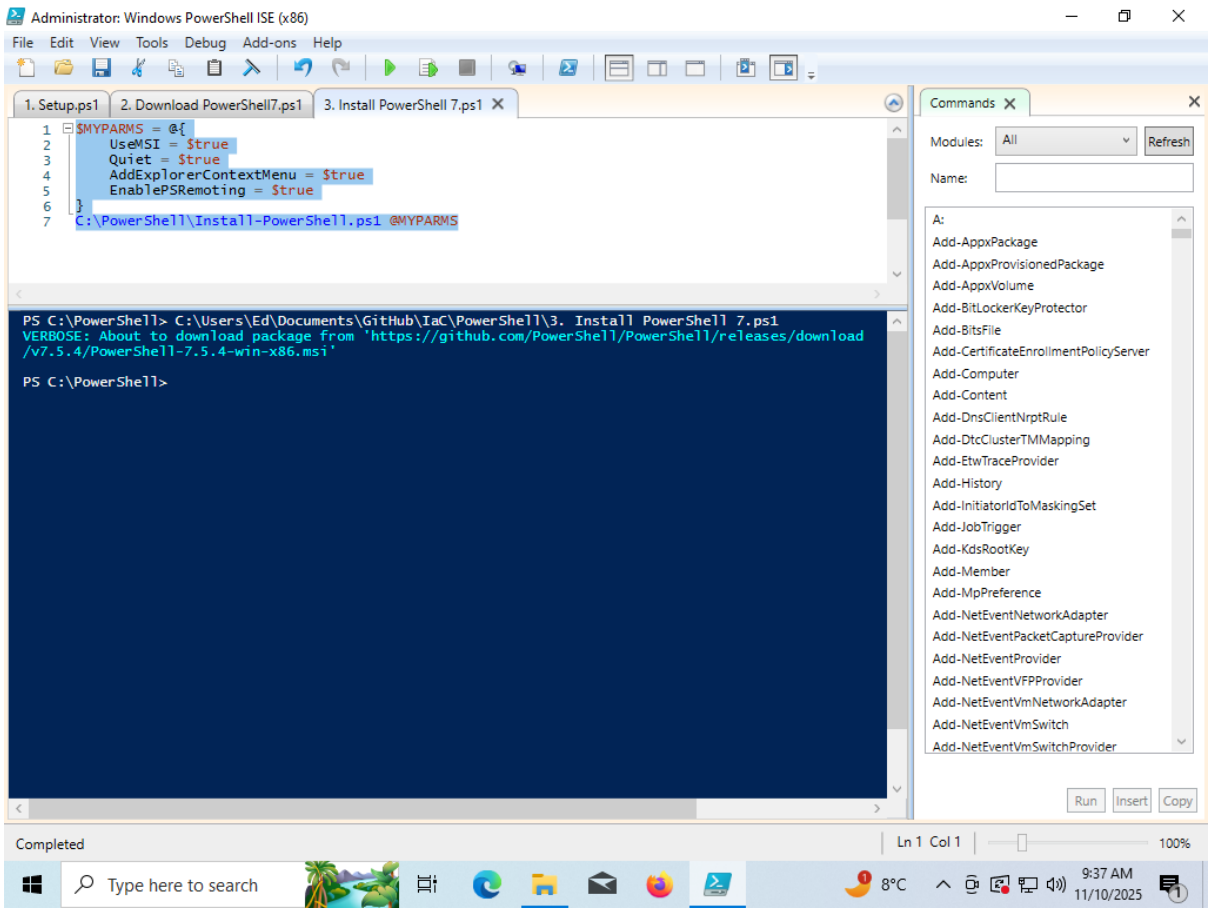

Figure 5 – Result of 3.Install PowerShell7.ps

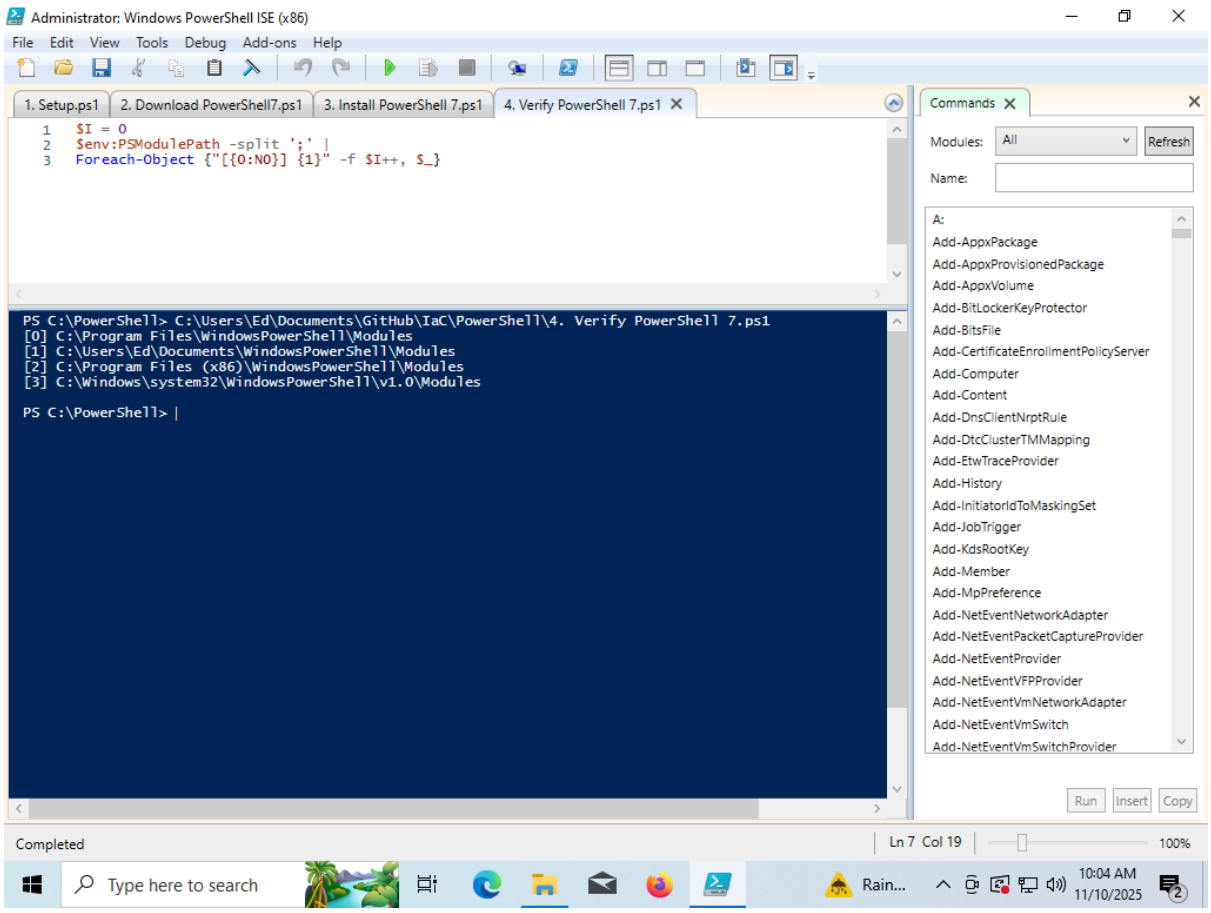

Figure 6 – Result of Verify PowerShell7.ps

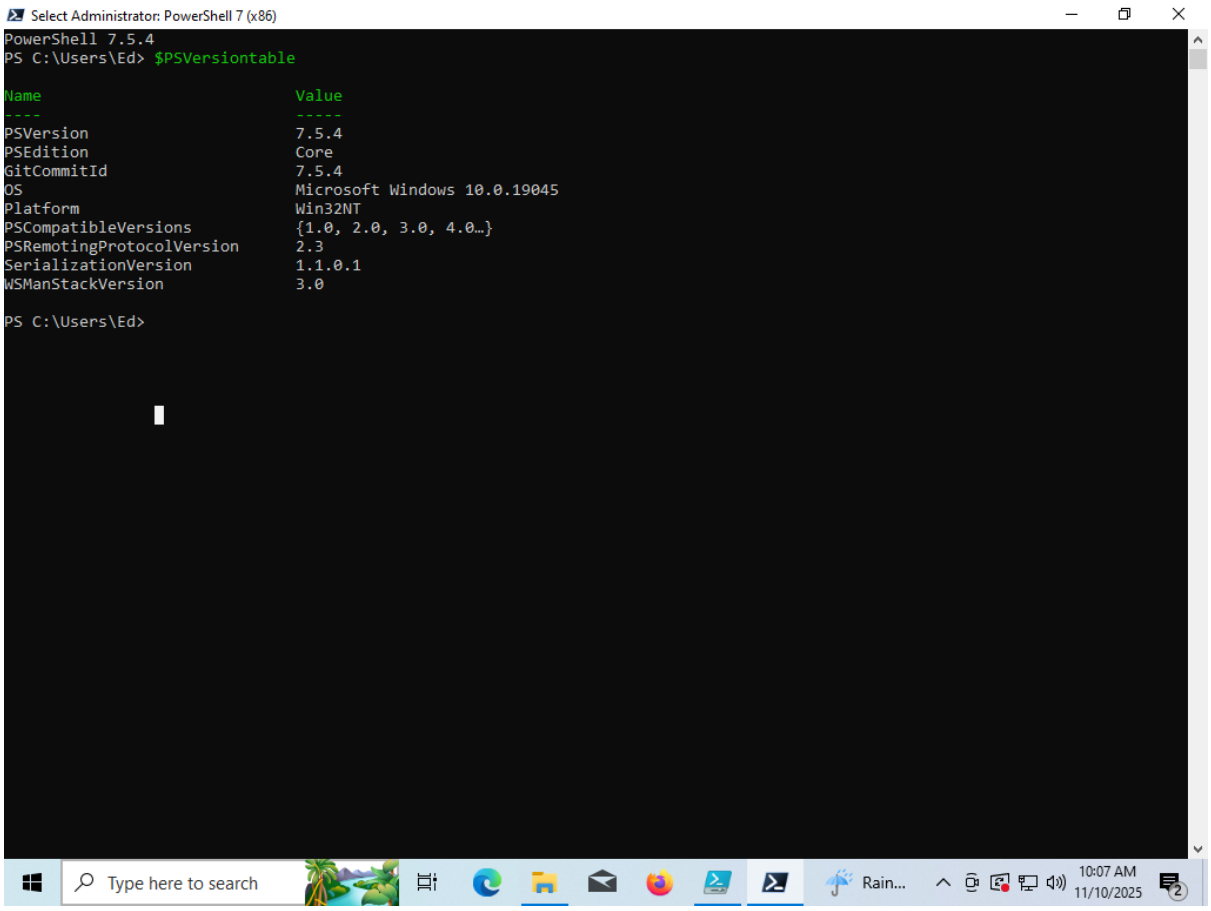

Figure 7 – Result of 4. Verify PowerShell7.ps1

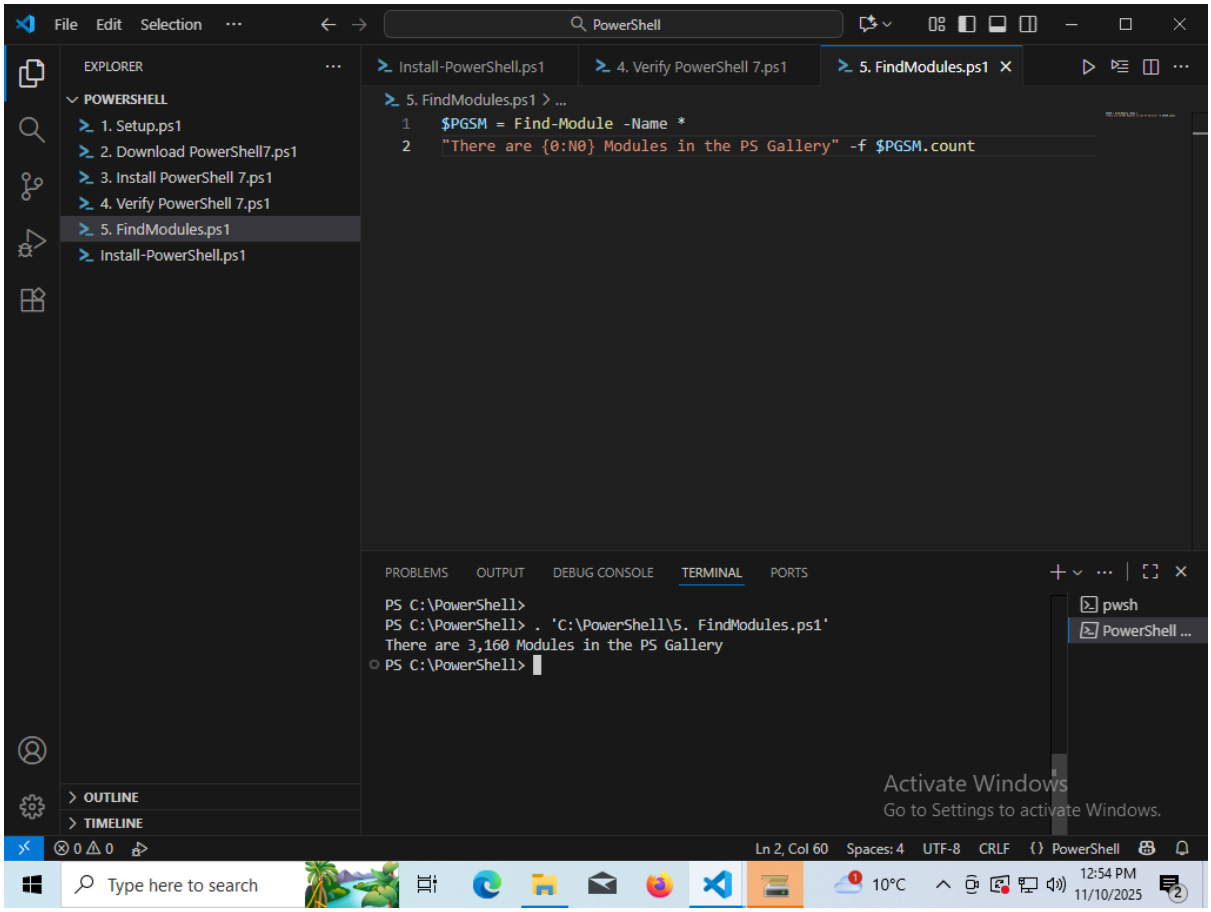

Figure 8 – Result of 5. FindModules.ps1

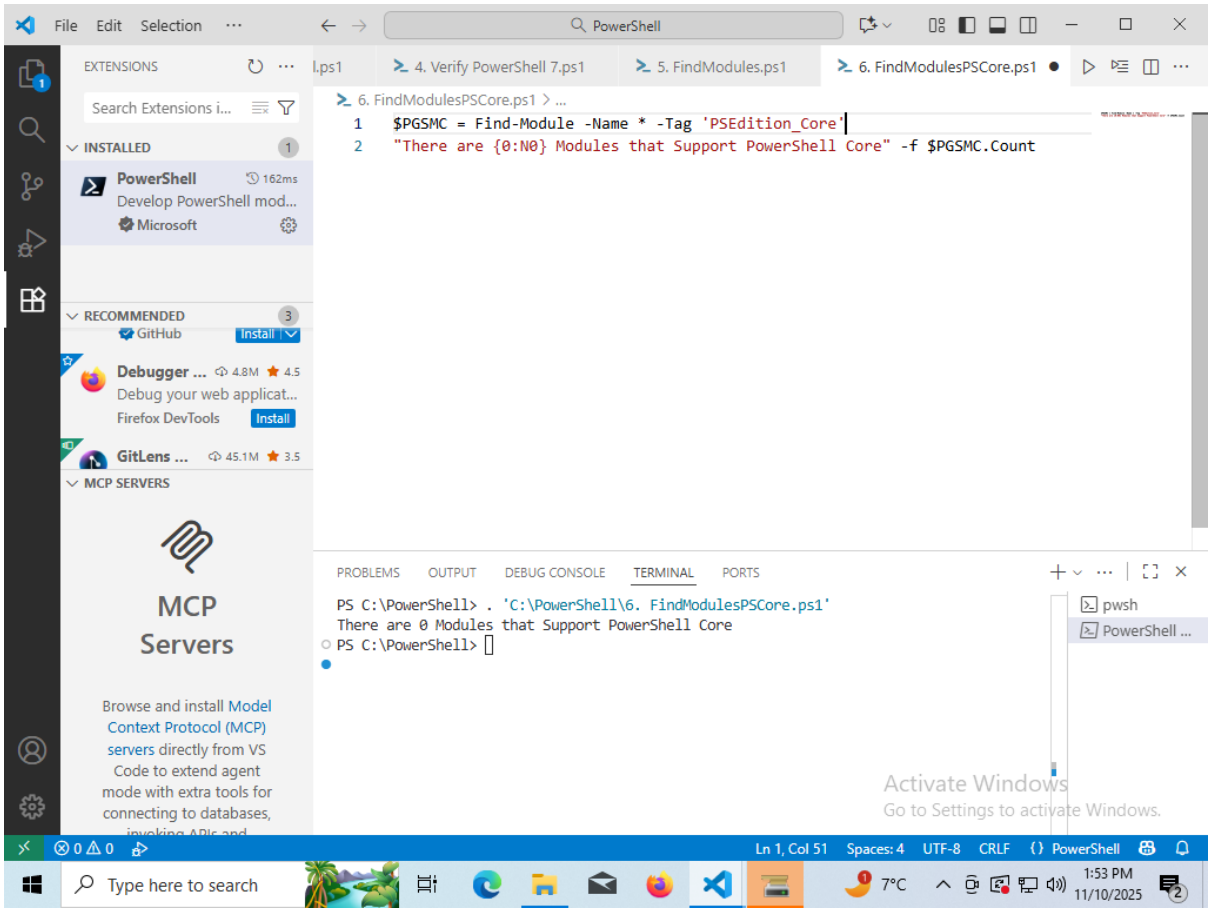

Figure 9 – Result of 6. FindModulesPSCore.ps1

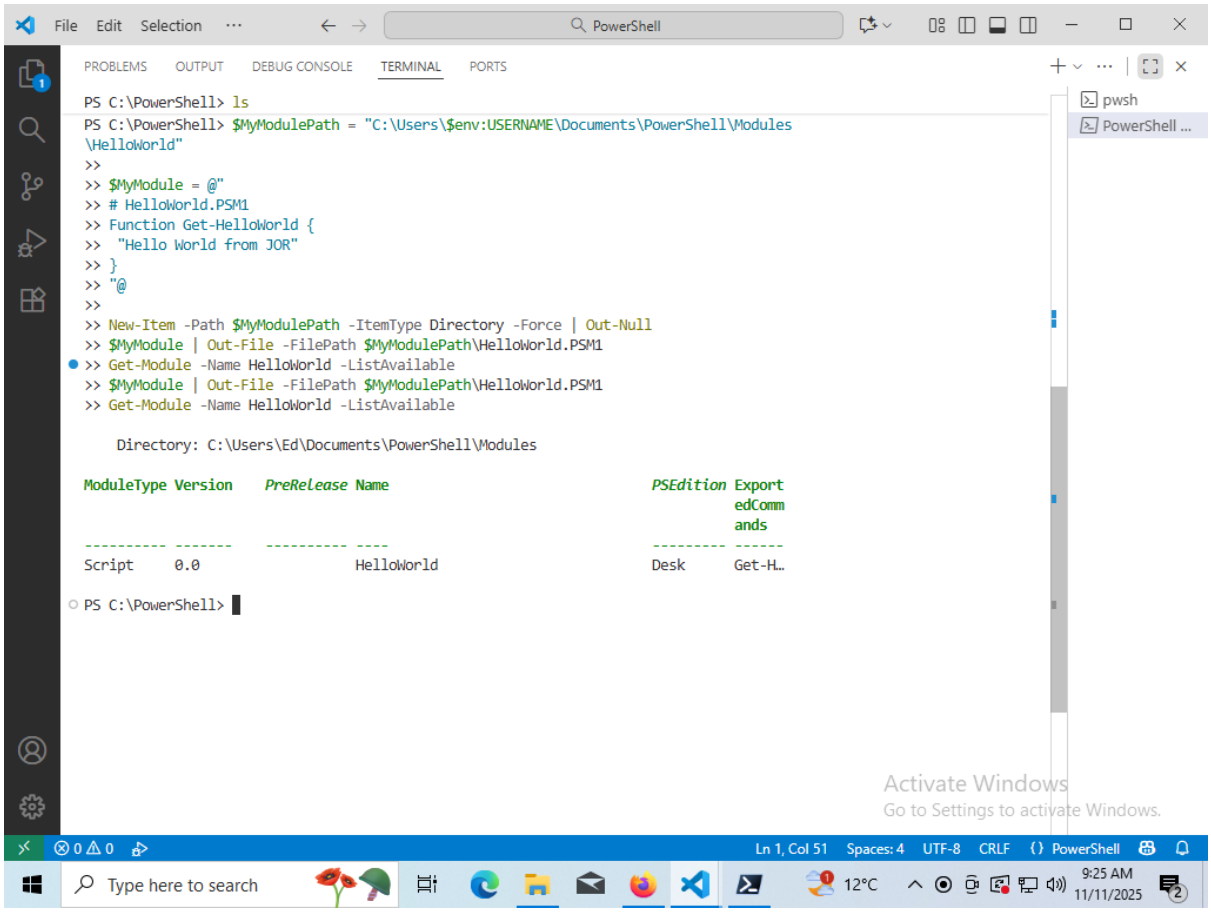

Figure 10 – PowerShell Module Code Result

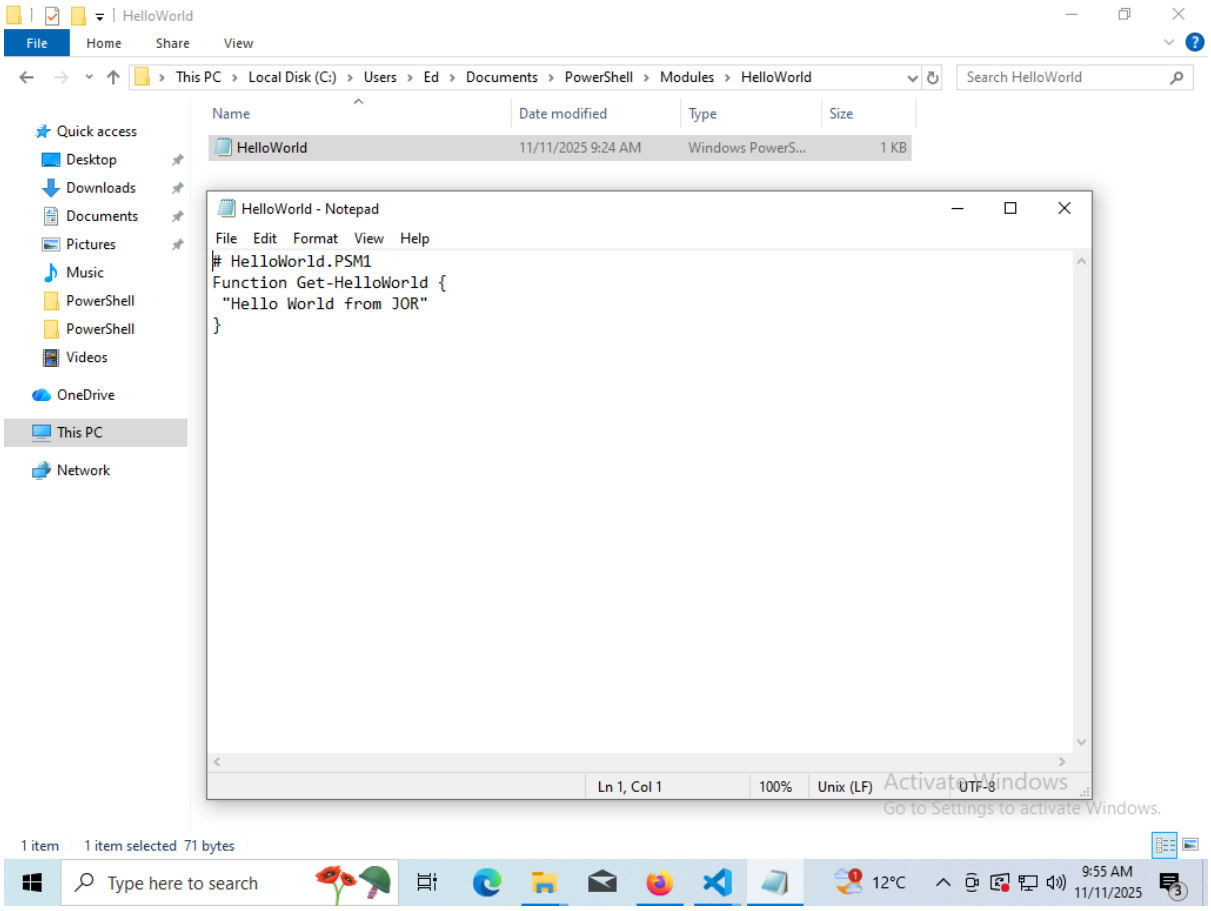

Figure 11 – PowerShell HelloWorld Module

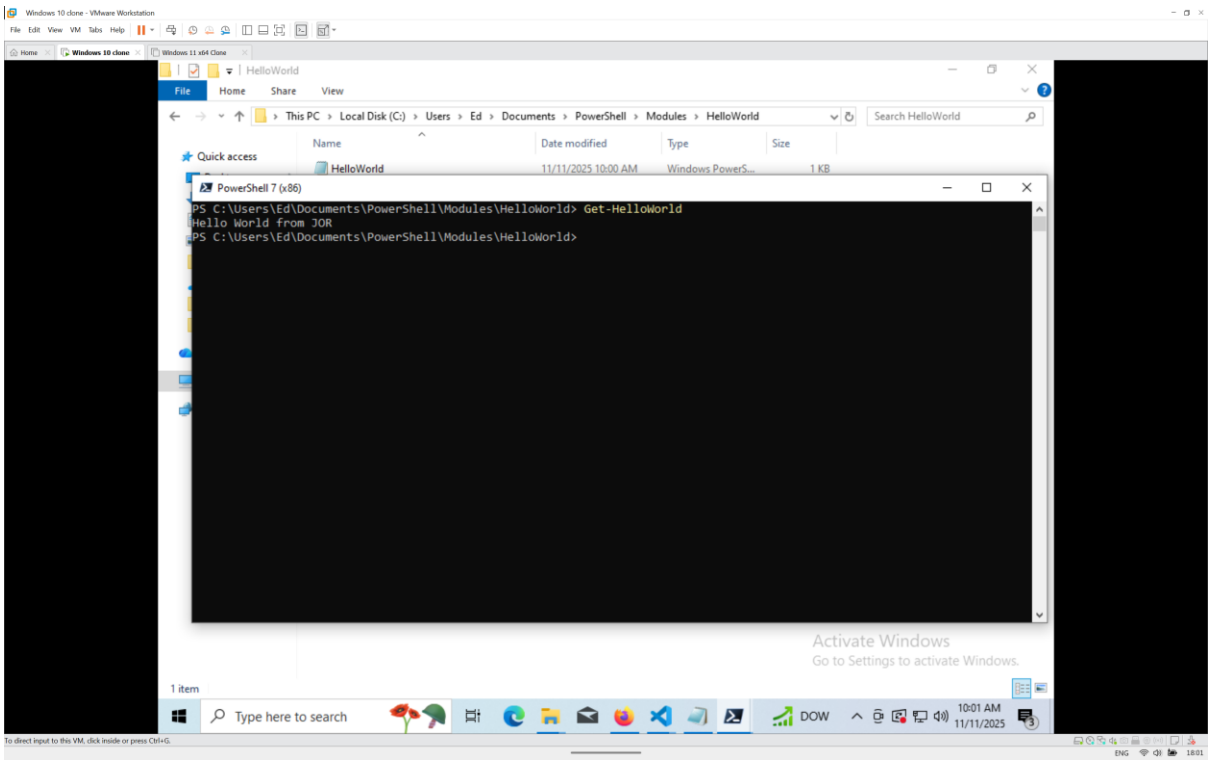

Figure 12 – Execution of HelloWorld Module

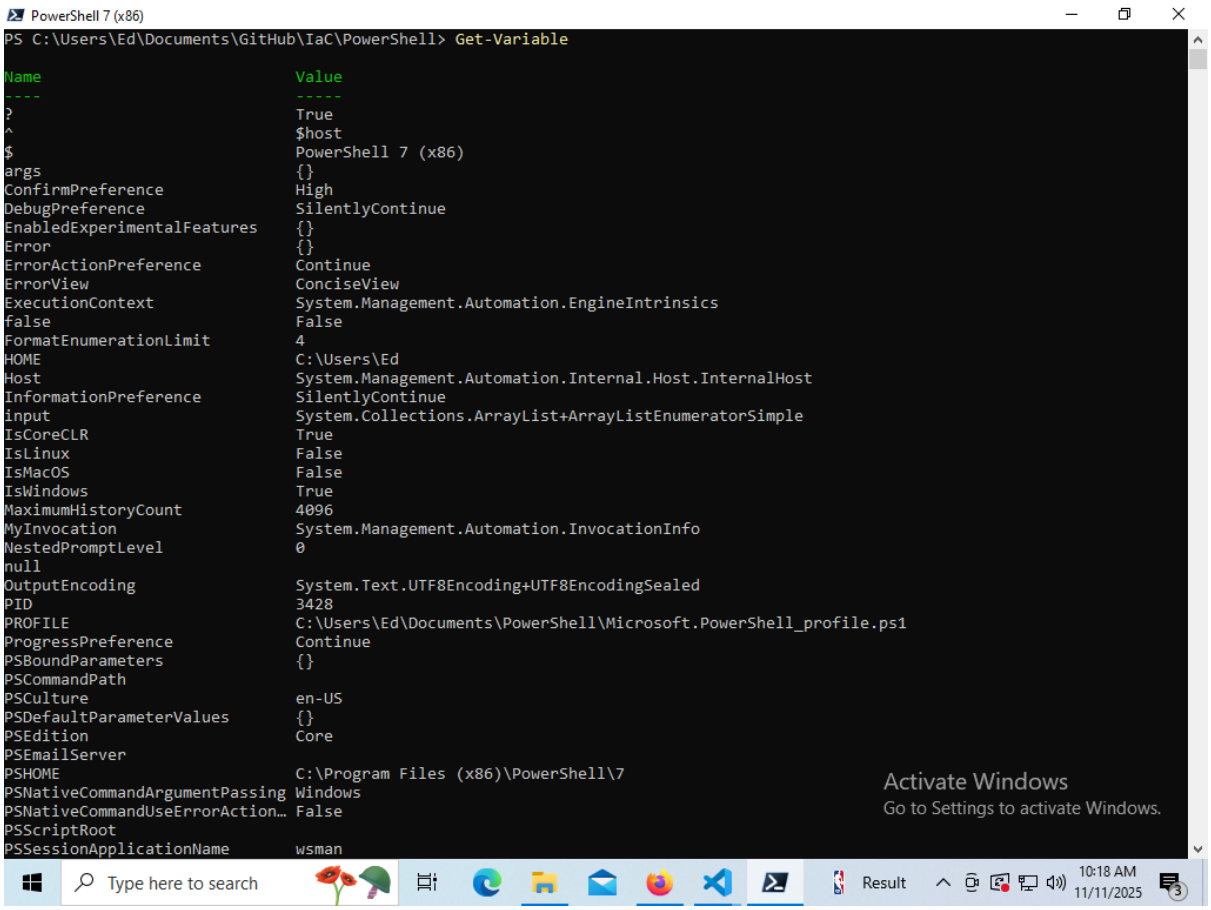

Figure 13 – Result of Get-Variable

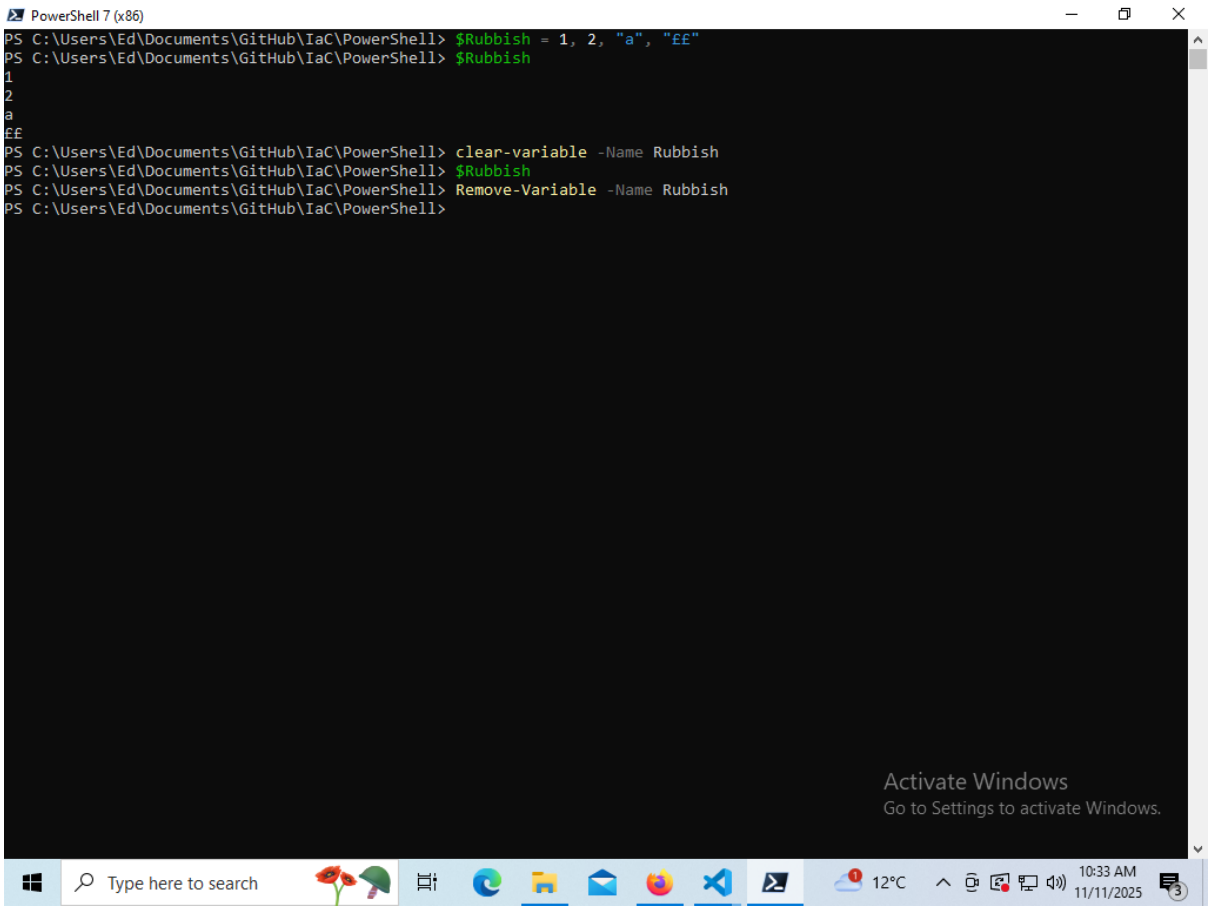

Figure 14 - $Rubbish Variable operations

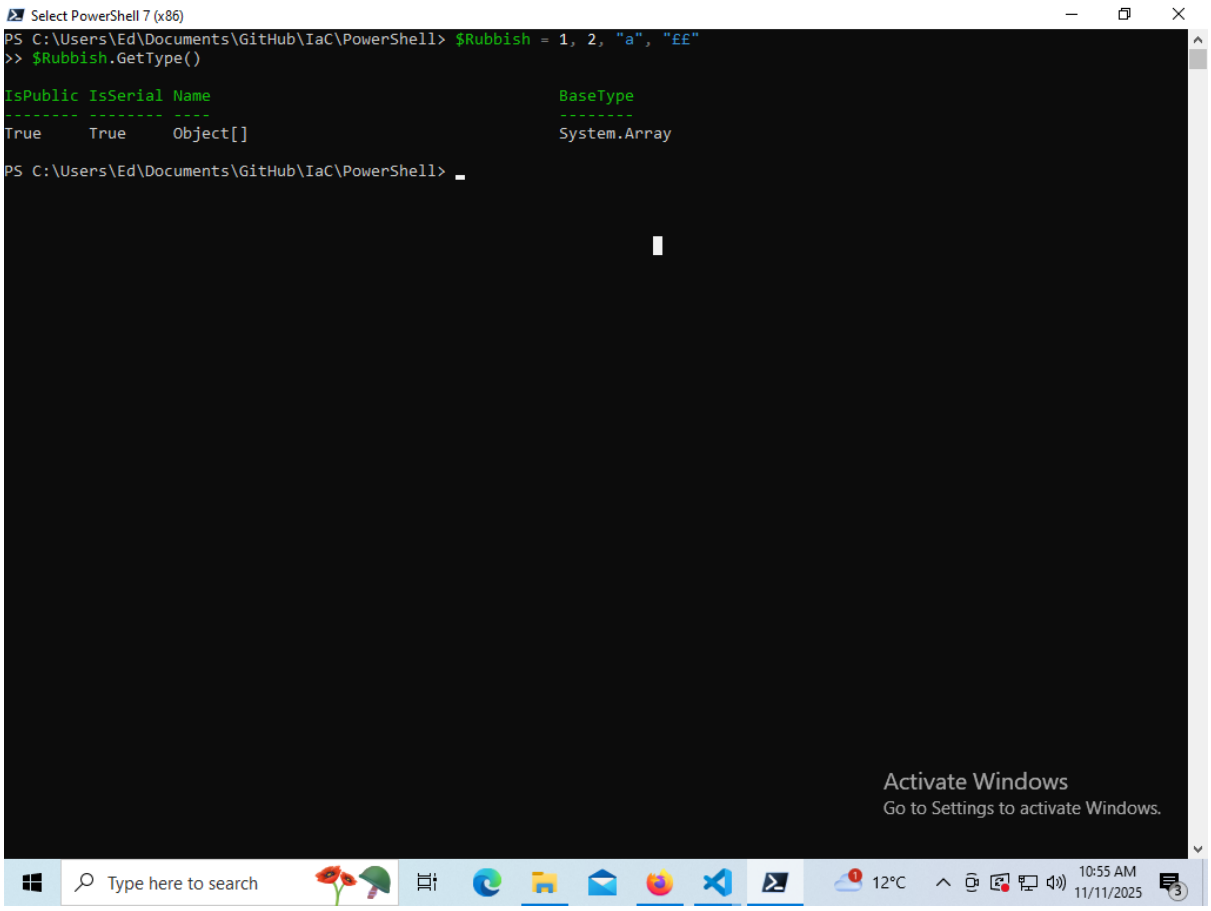

Figure 15 – Result of GetType of a variable

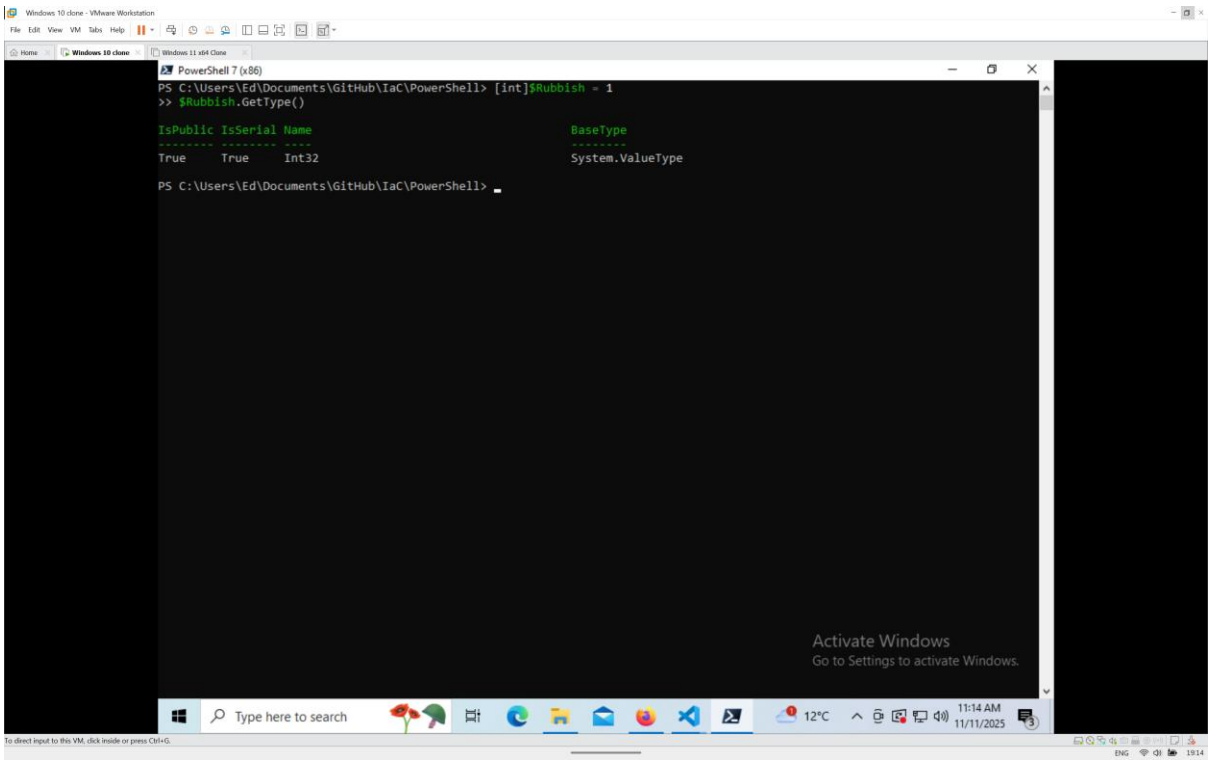

Figure 16 – Result from Casting

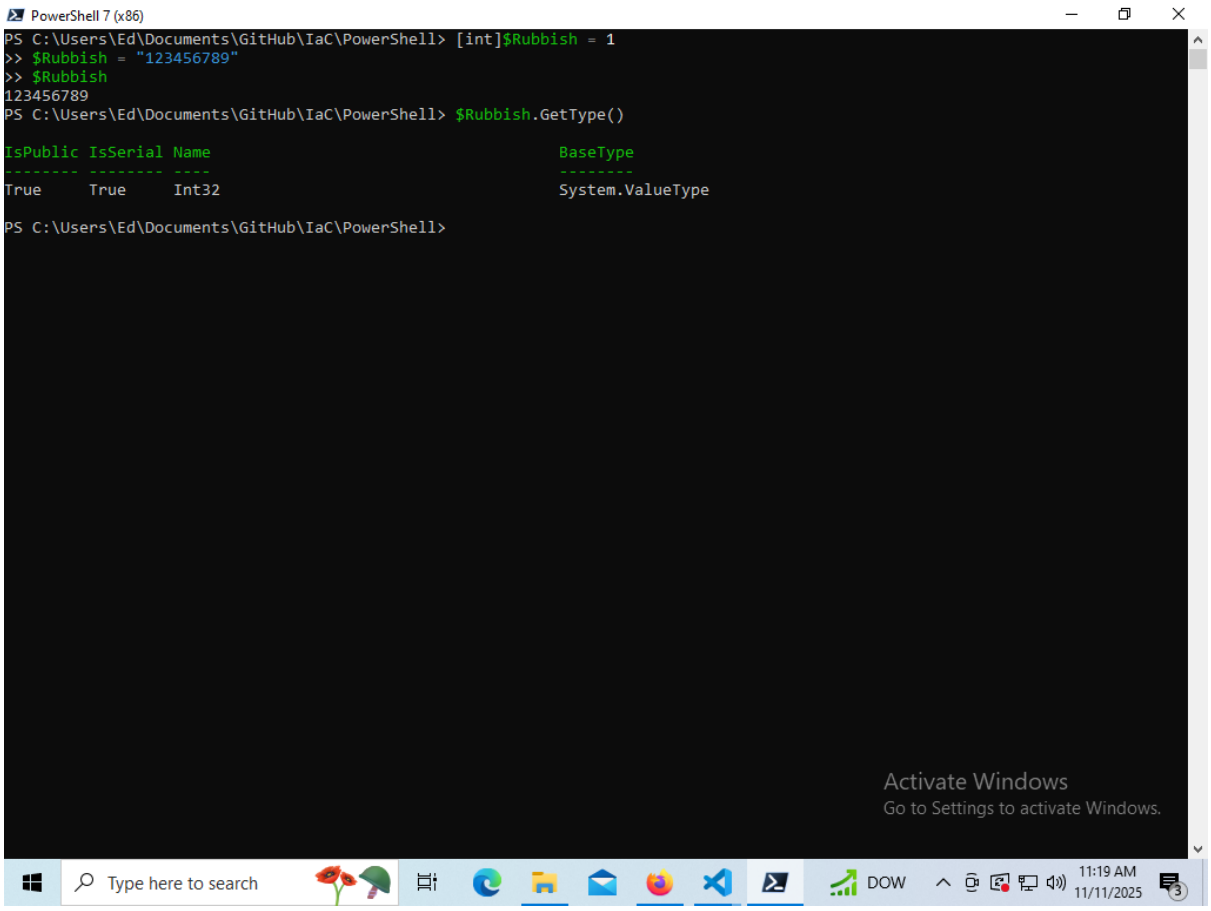

Figure 17 – Automatic Cast Conversion

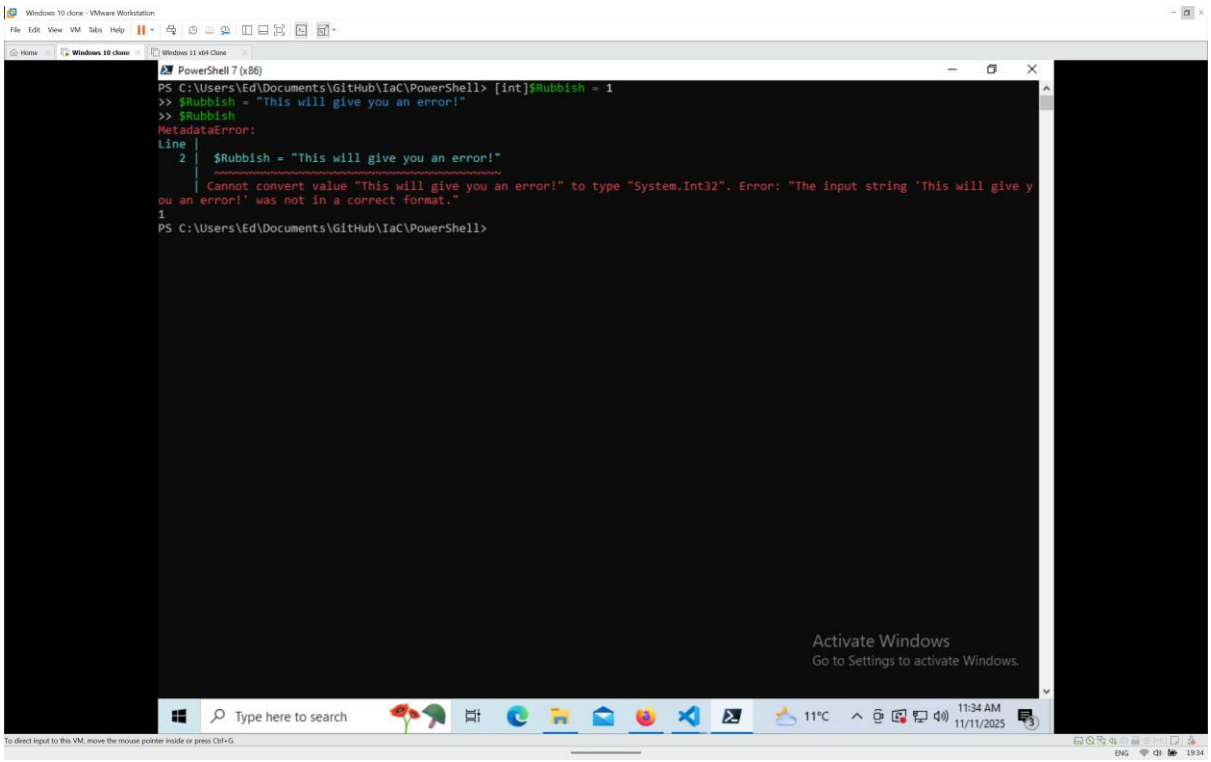

Figure 18 – Unsuccessful Cast Conversion

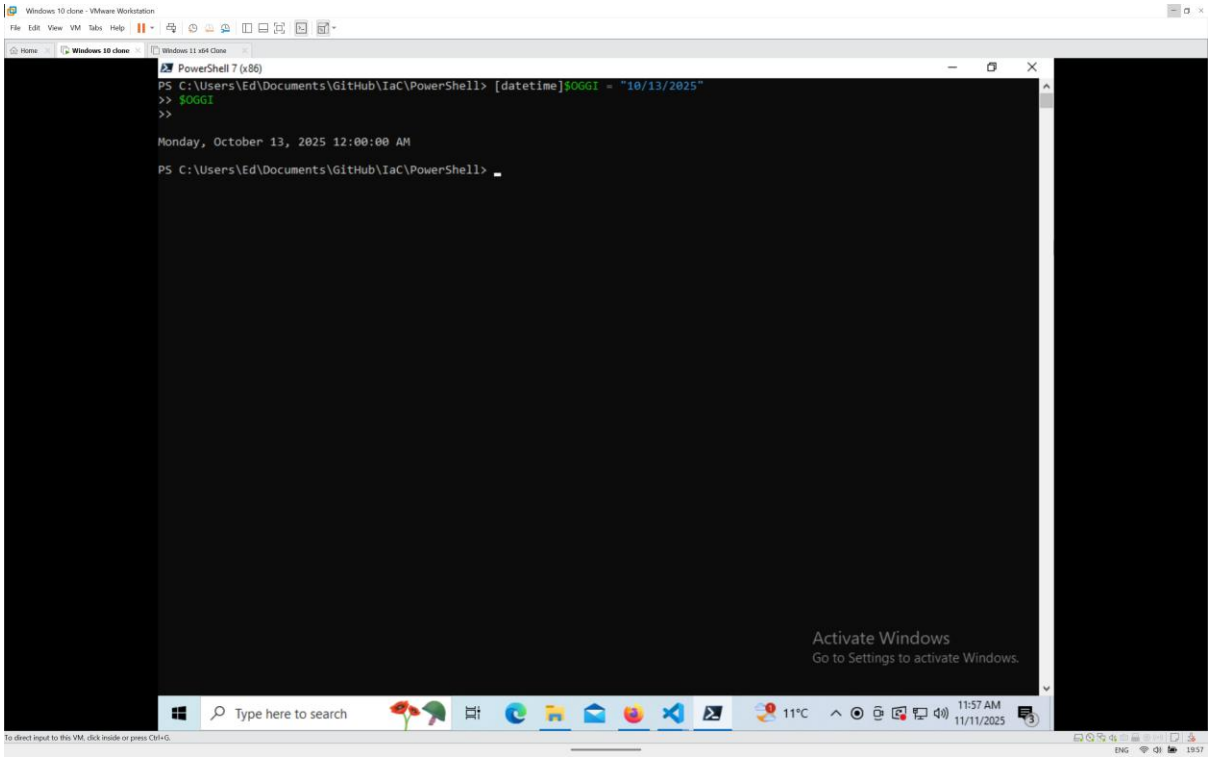

Figure 19 – datetime cast

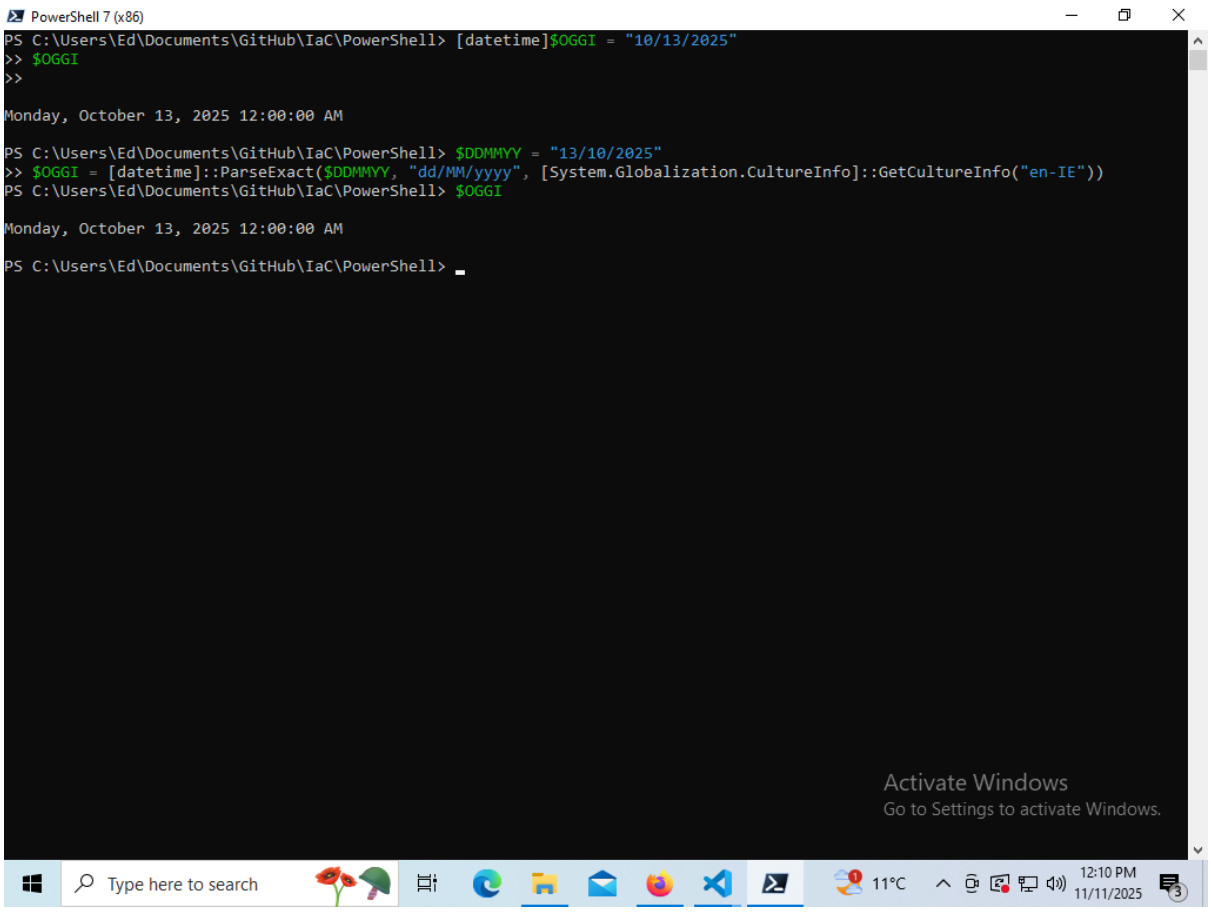

Figure 20 – DD/MM/YY datetime

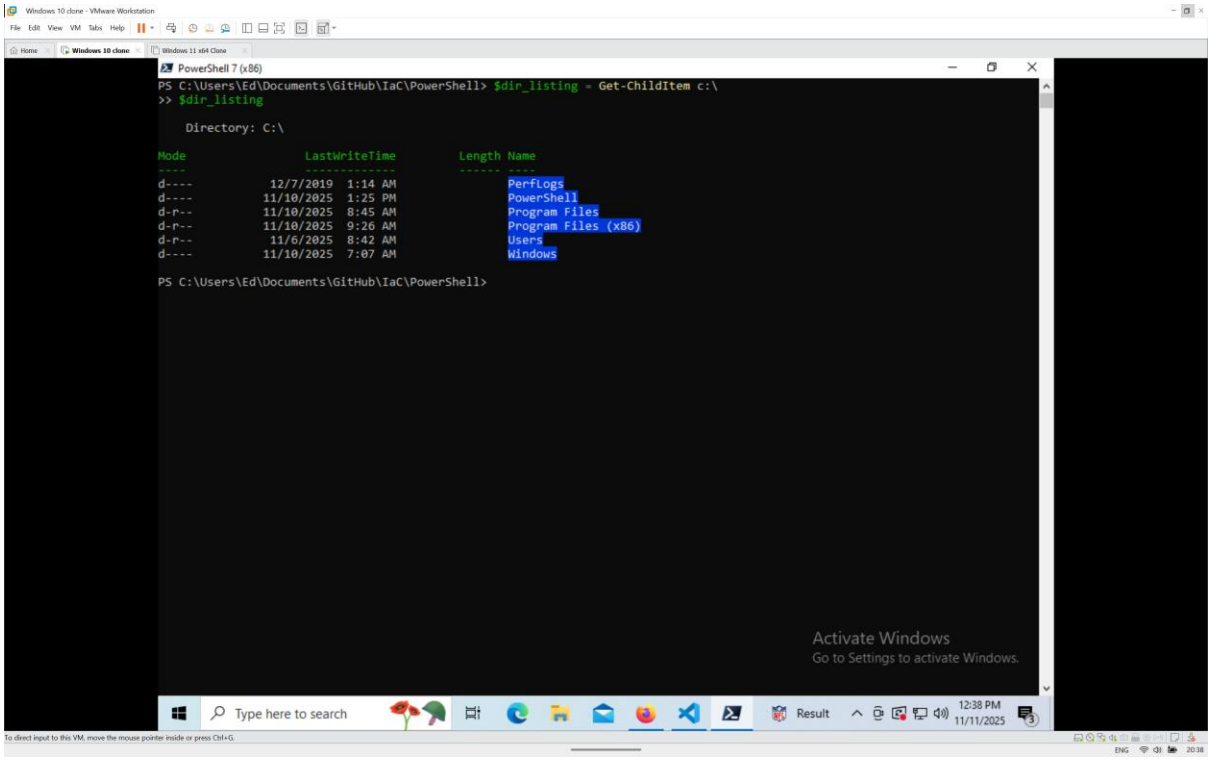

Figure 21 – Result of saving output of command

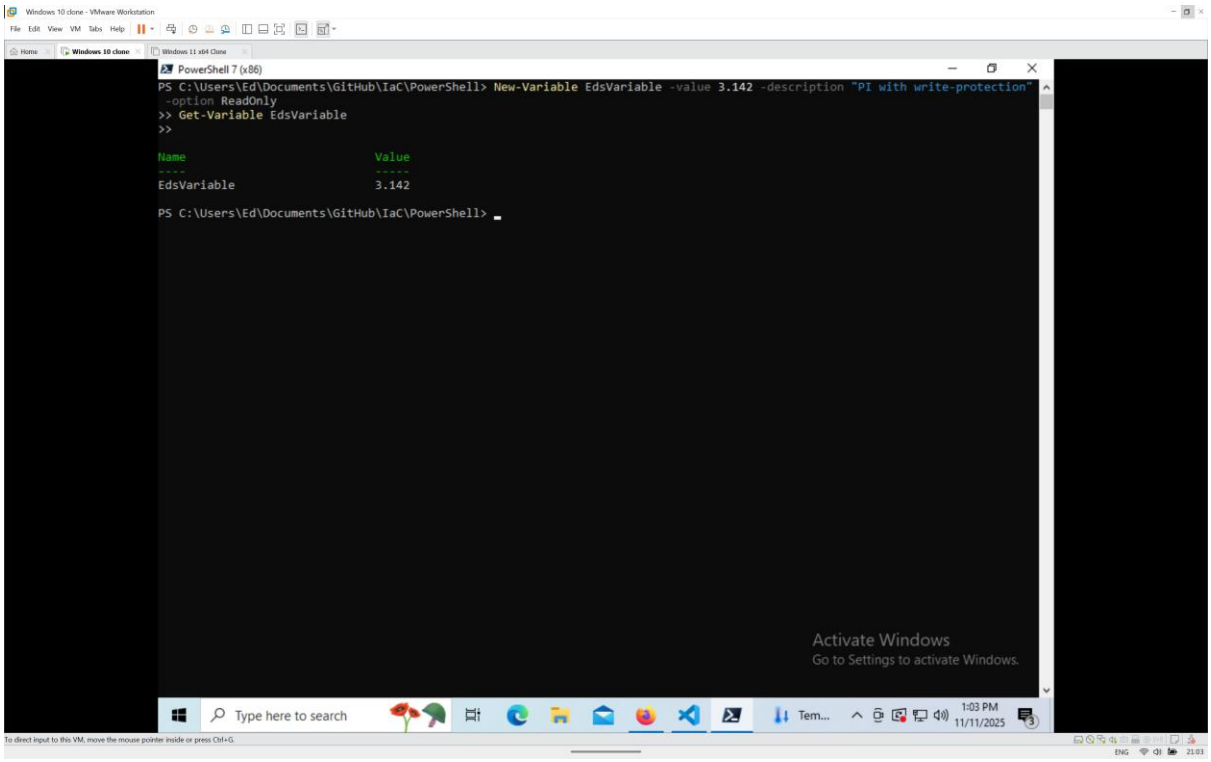

Figure 22 – Check if a variable exists

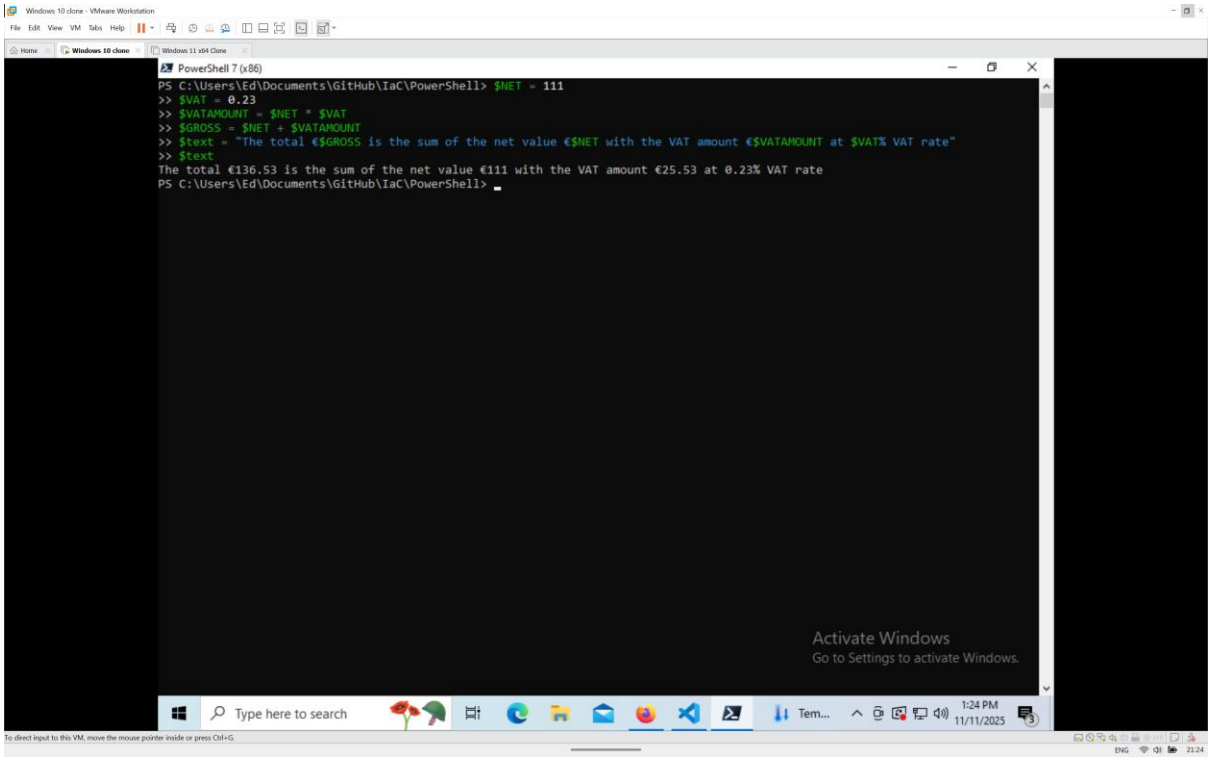

Figure 23 – Result of Tax Calculation

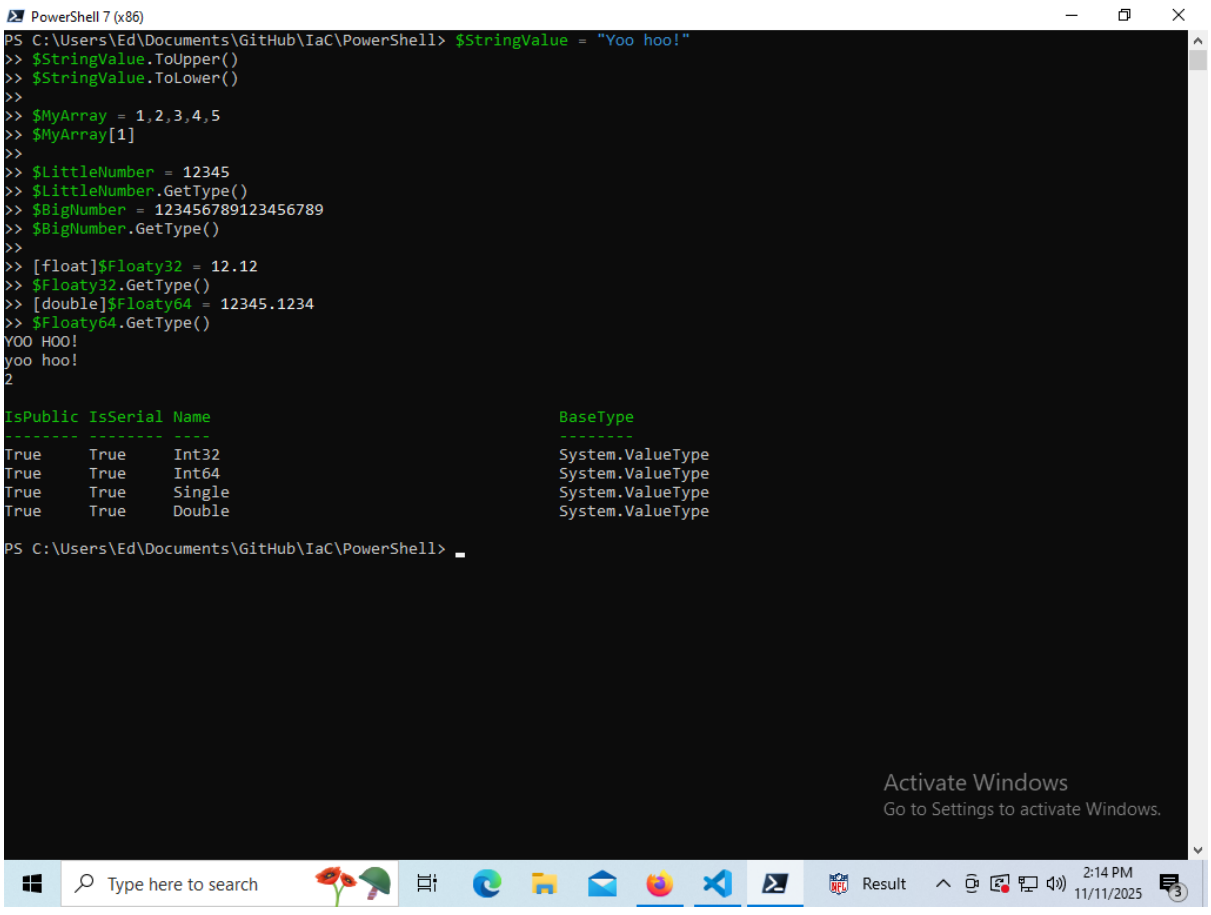

Figure 24 – Result for Types Exercises

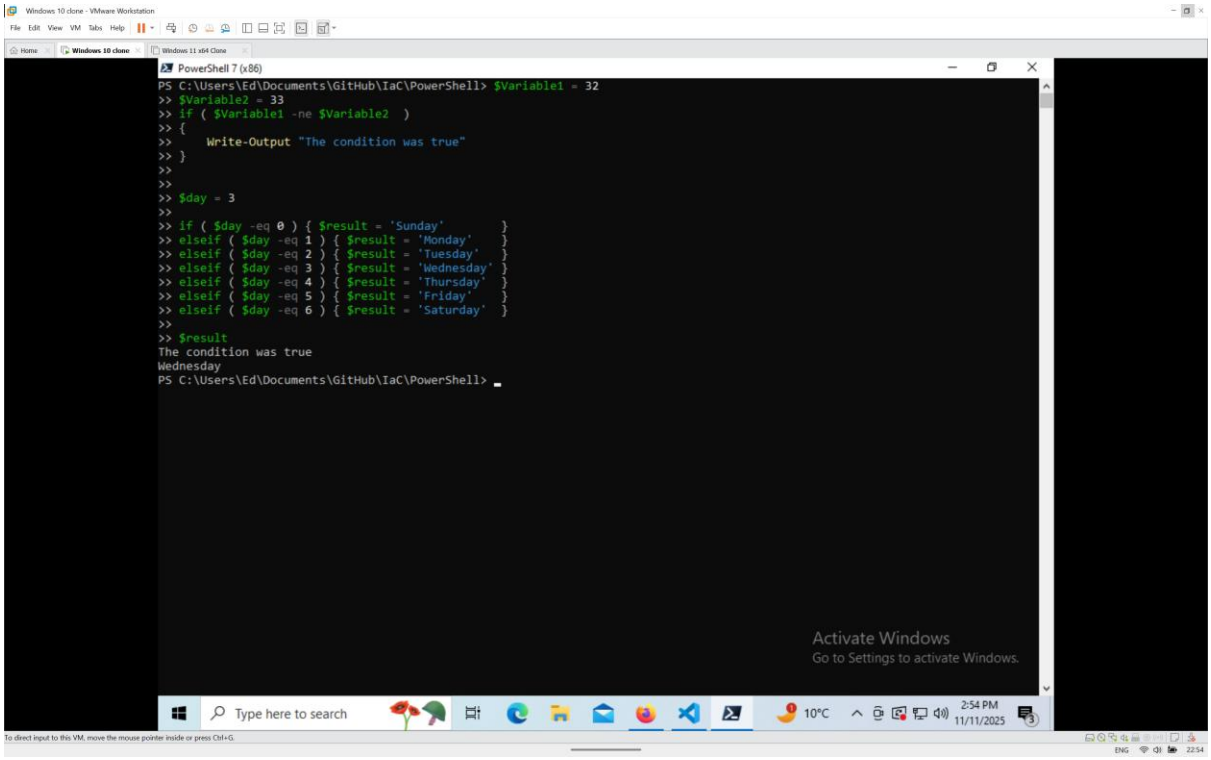

Figure 25 - Result for if and elseif statements

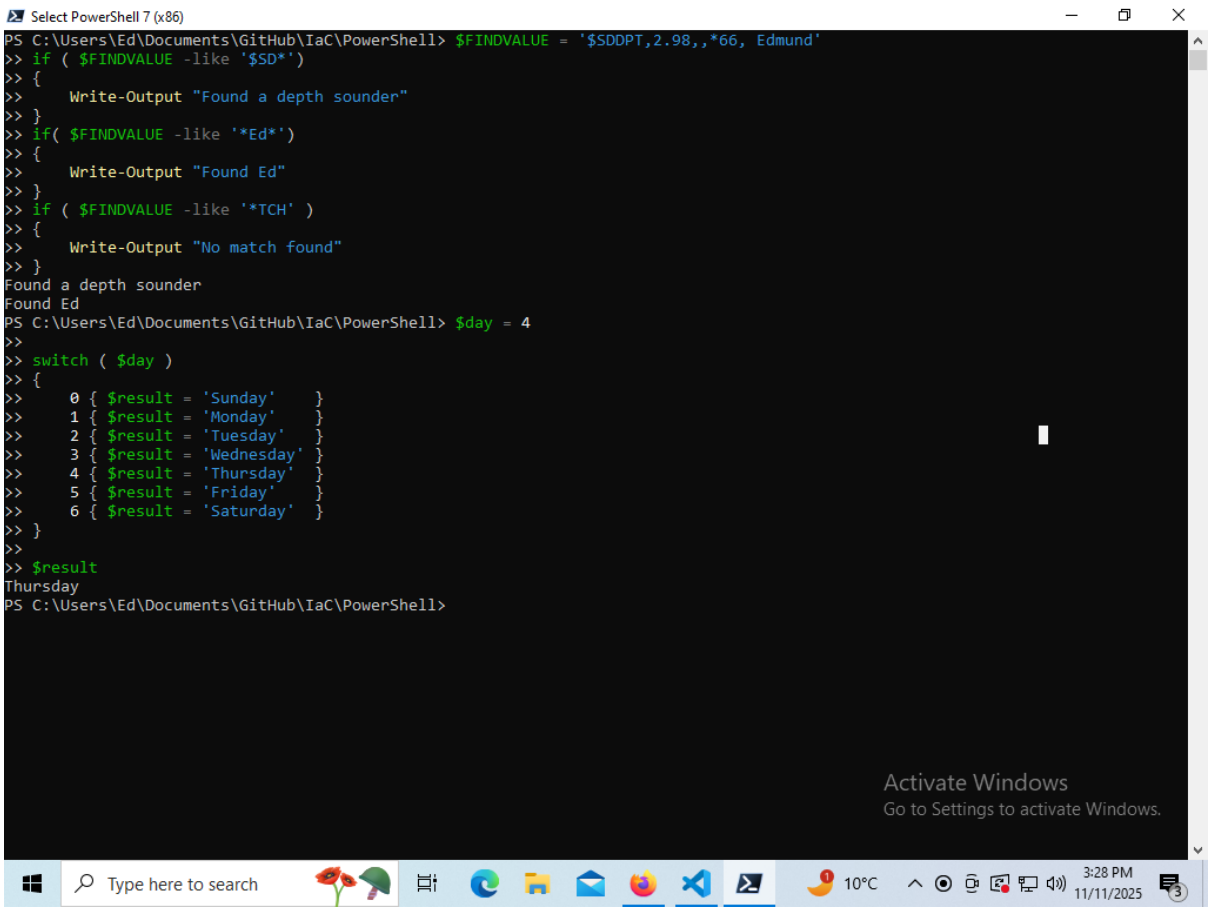

Figure 26 - Result for -like and Switch

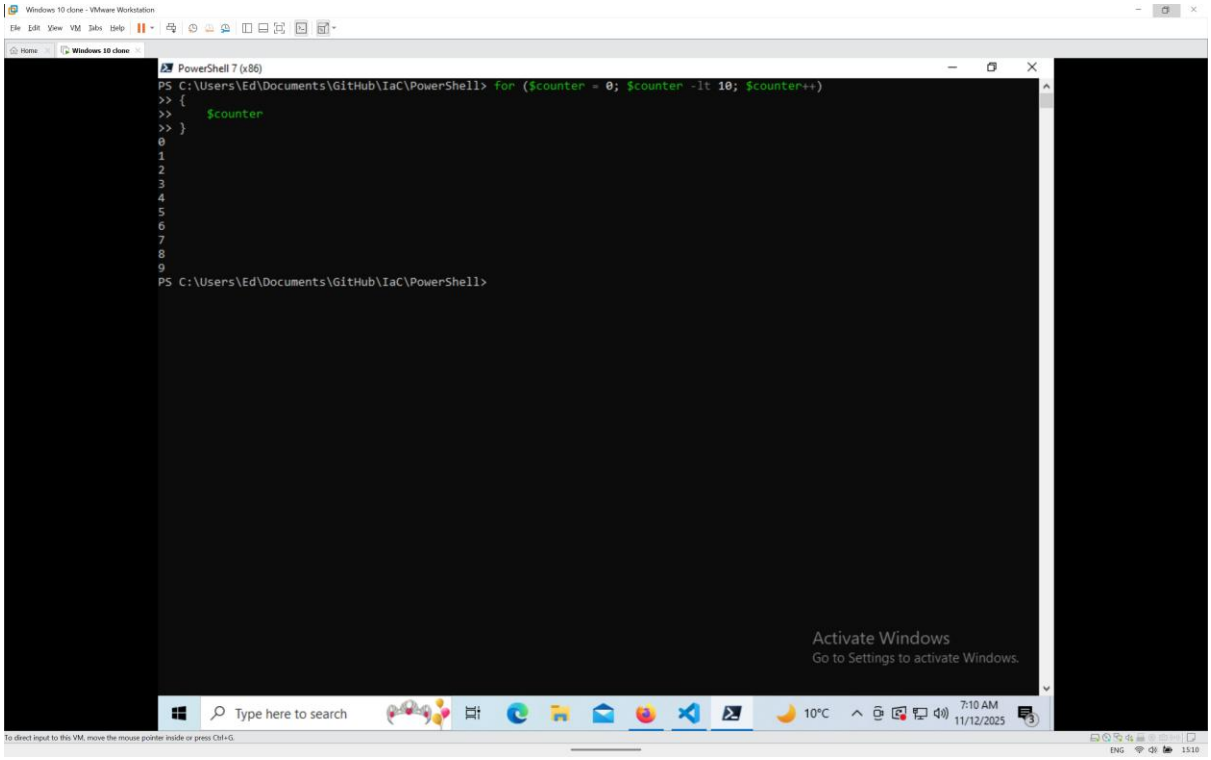

Figure 27 - Result for For Loop

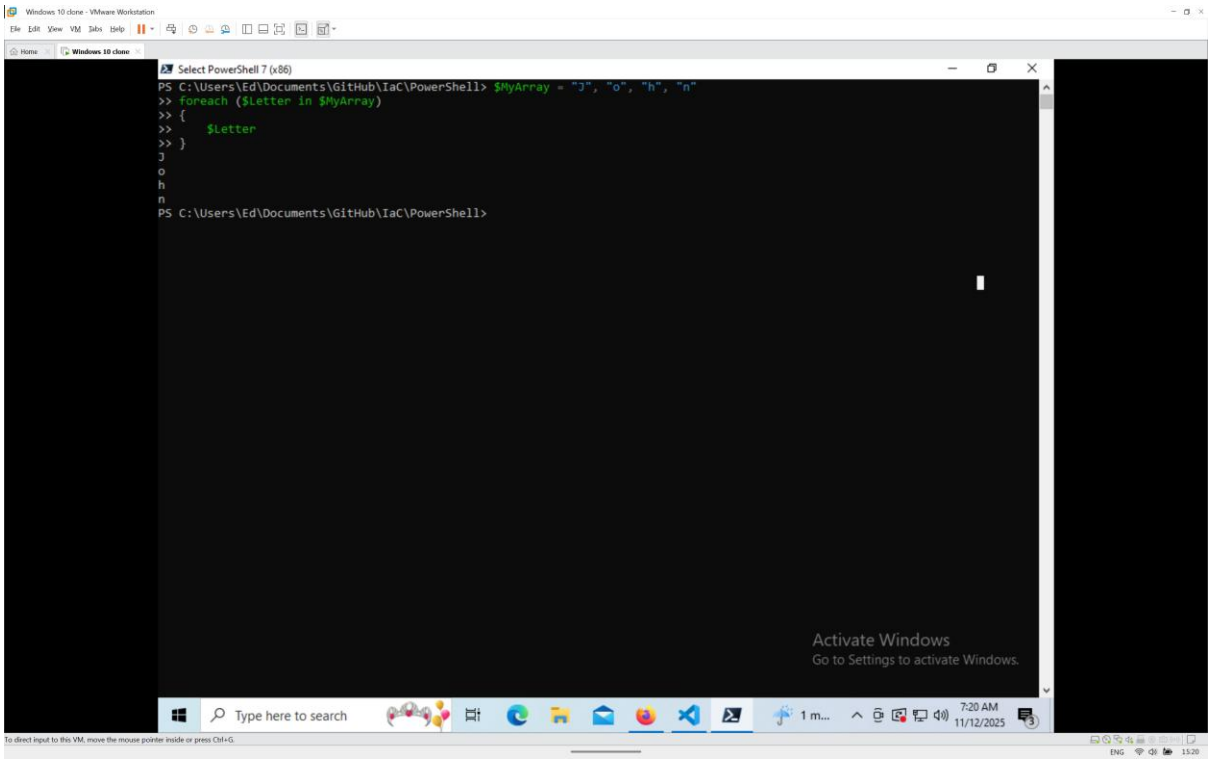

Figure 28- Result of ForEach Loop

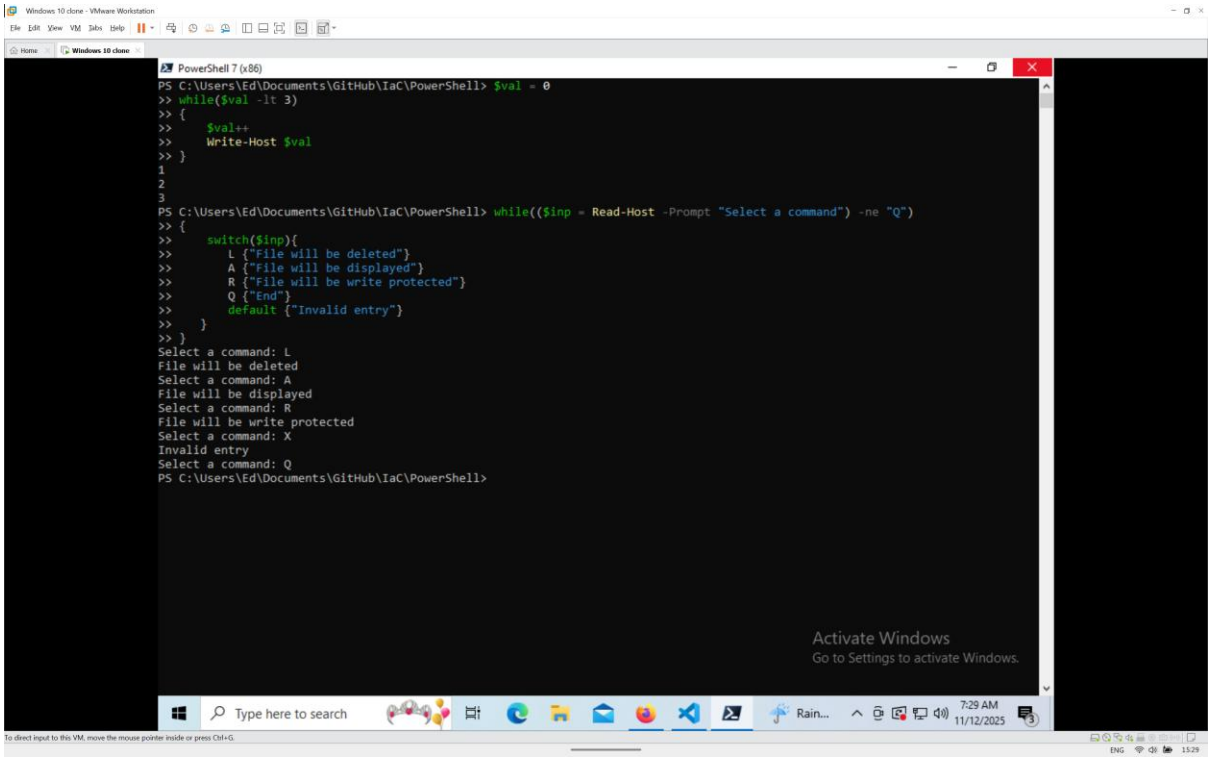

Figure 29 - Results for While Loop code

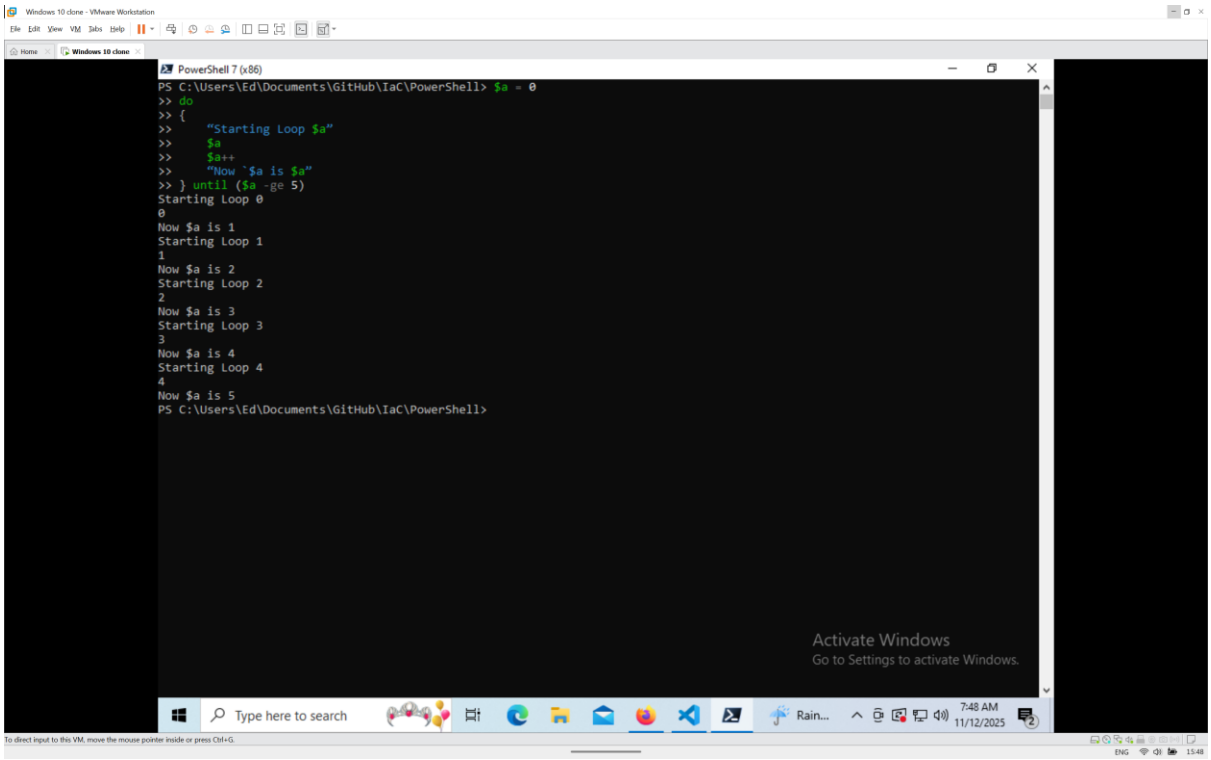

Figure 30 - Do Until Loop Results

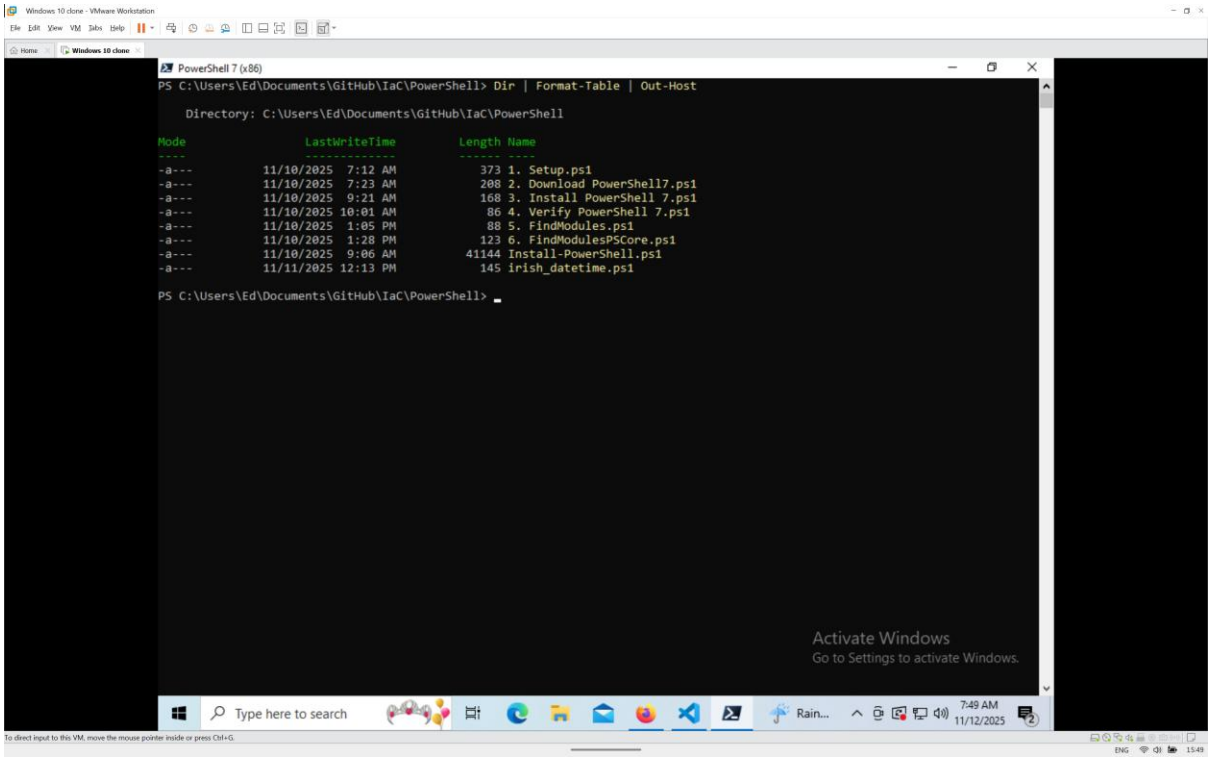

Figure 31 - Result from Piping